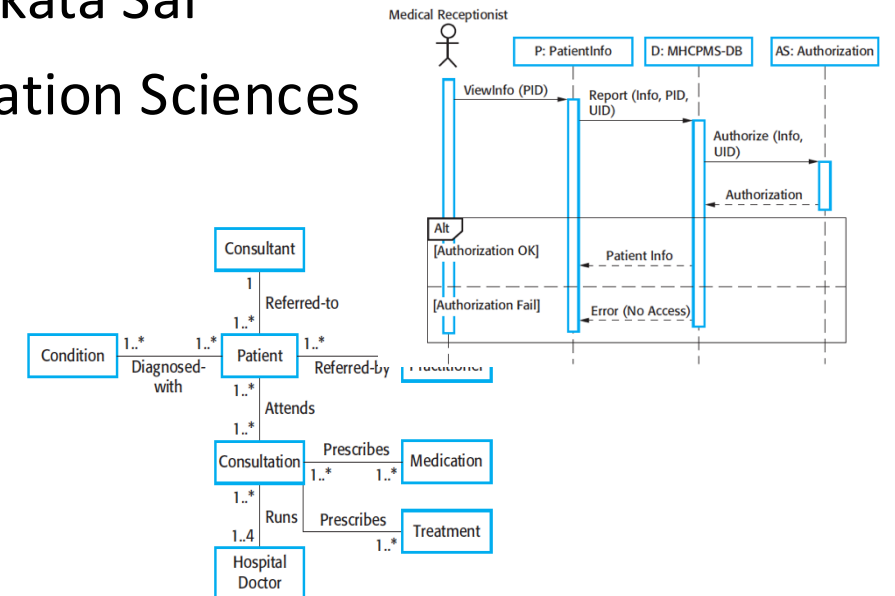
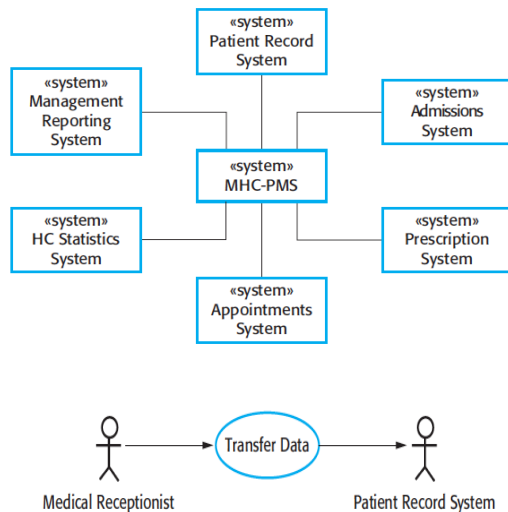


COSC 612/ AIT 624

System Modeling

Dr. Akshita Maradapu Vera Venkata Sai
Department of Computer and Information Sciences
Towson University



System Modeling

- System modeling is the process of developing abstract models of a system,
 - Each model presenting a different view or perspective of that system.
- System modeling means representing a system using some kind of graphical notation,
 - almost always based on notations in the Unified Modeling Language (UML).

System Modeling

- **Why System modelling?**

- Models helps the analyst to understand the functionality of the system
- Models are used to communicate with customers.
- Models are used during the requirements engineering process to help
 - derive the requirements for a system
- Models are used during the design process to describe the system to engineers
- Models are used to implement the system
- Models are used after implementation to document the system's structure and operation.

System Perspectives

- An **external perspective**,
 - where you model the context or environment of the system.
- An **interaction perspective**,
 - where you model the interactions between a system and its environment,
 - or between the components of a system.
- A **structural perspective**,
 - where you model the organization of a system
 - or the structure of the data that is processed by the system.
- A **behavioral perspective**,
 - where you model the dynamic behavior of the system and how it responds to events.

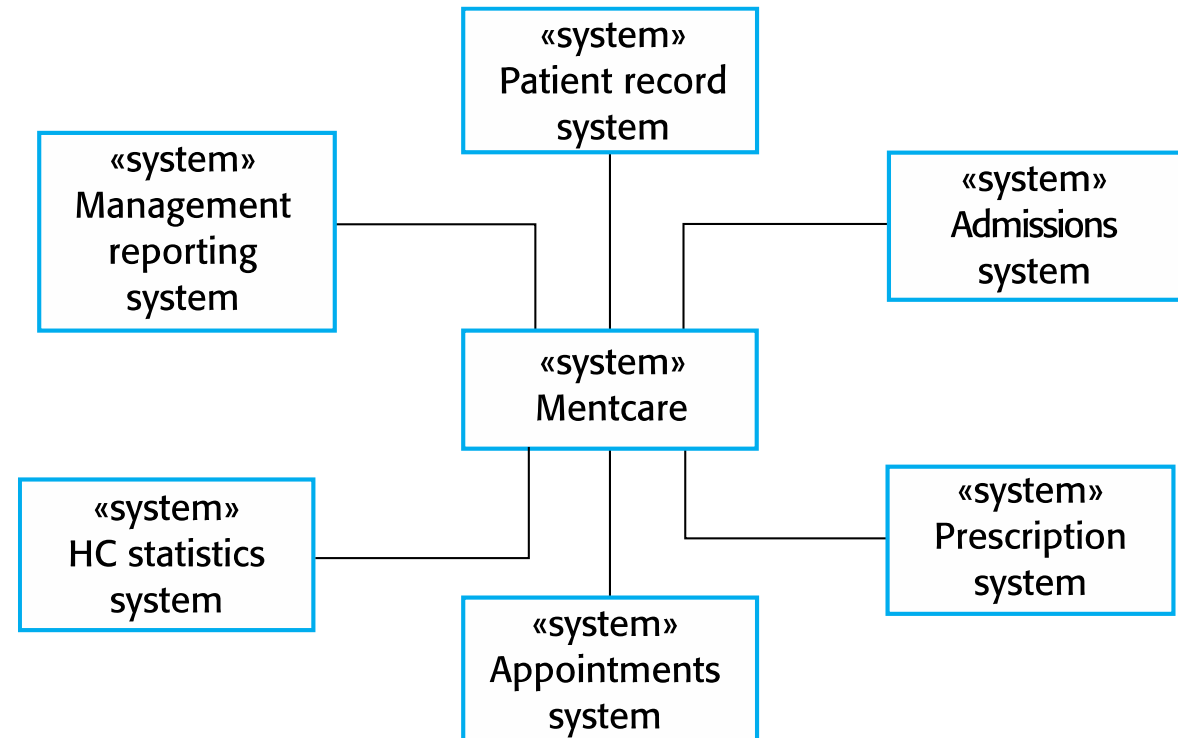
Context Model

Context models

- Context models are used to
 - illustrate the operational context of a system –
 - they show what lies outside the system boundaries.
- **Social and organisational concerns** may affect the decision on where to position system boundaries.
- **Architectural models** show the system and its **relationship with other systems**.

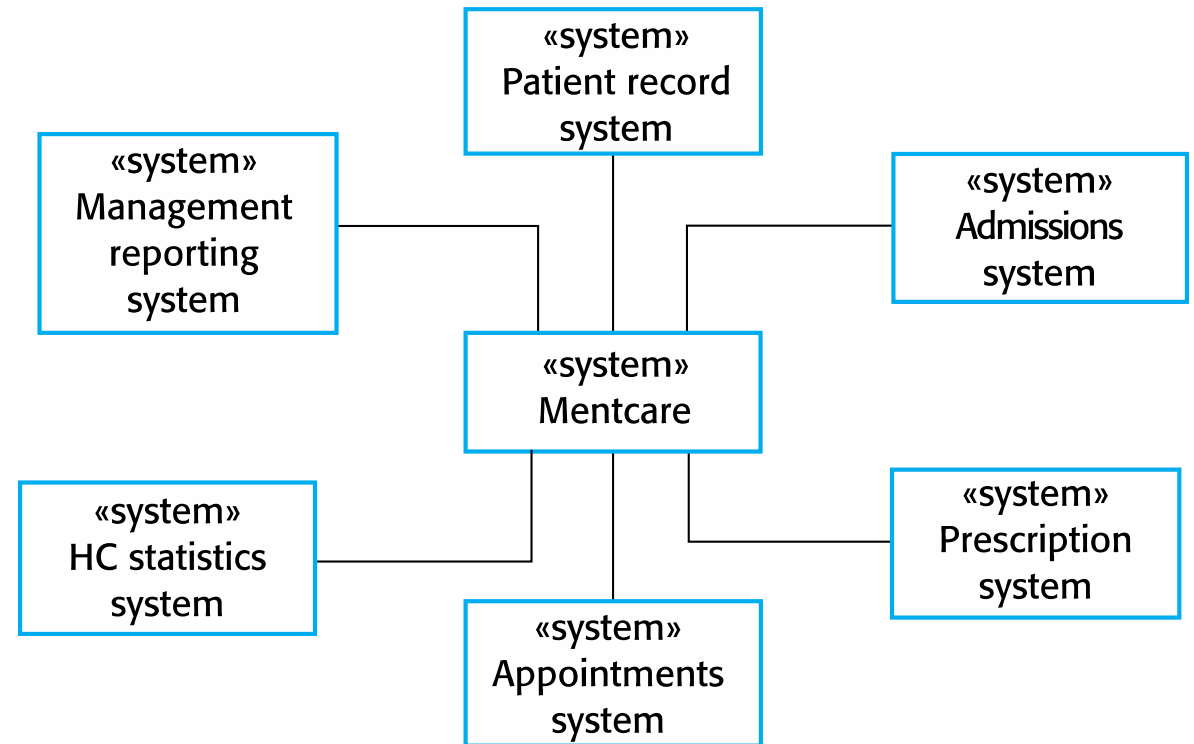
The context of the Mentcare system

- **Context models** simply show the other systems in the environment,
 - not how the system being developed is **used** in that environment.



The context of the Mentcare system

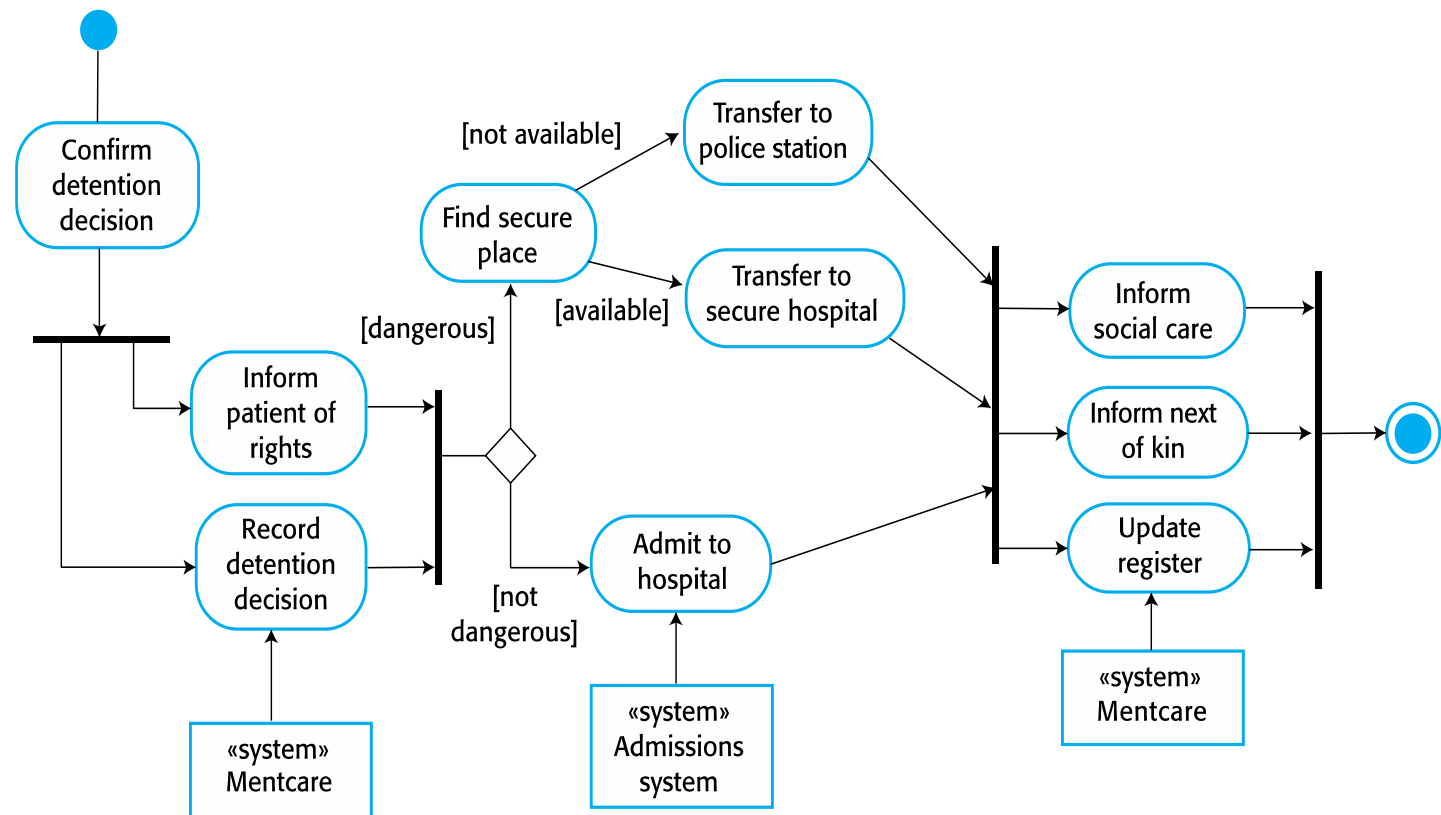
- **System boundaries** are established to:
 - define what is inside and what is outside the system.
 - show other systems that are used or depend on the system being developed.



Process Perspective

- **Process Models**

- reveal how the system being developed is used in broader business processes.
- UML activity diagrams may be used to define business process models.



show the **activities** involved in a process or in **data processing**

Unified Modeling Language (UML)

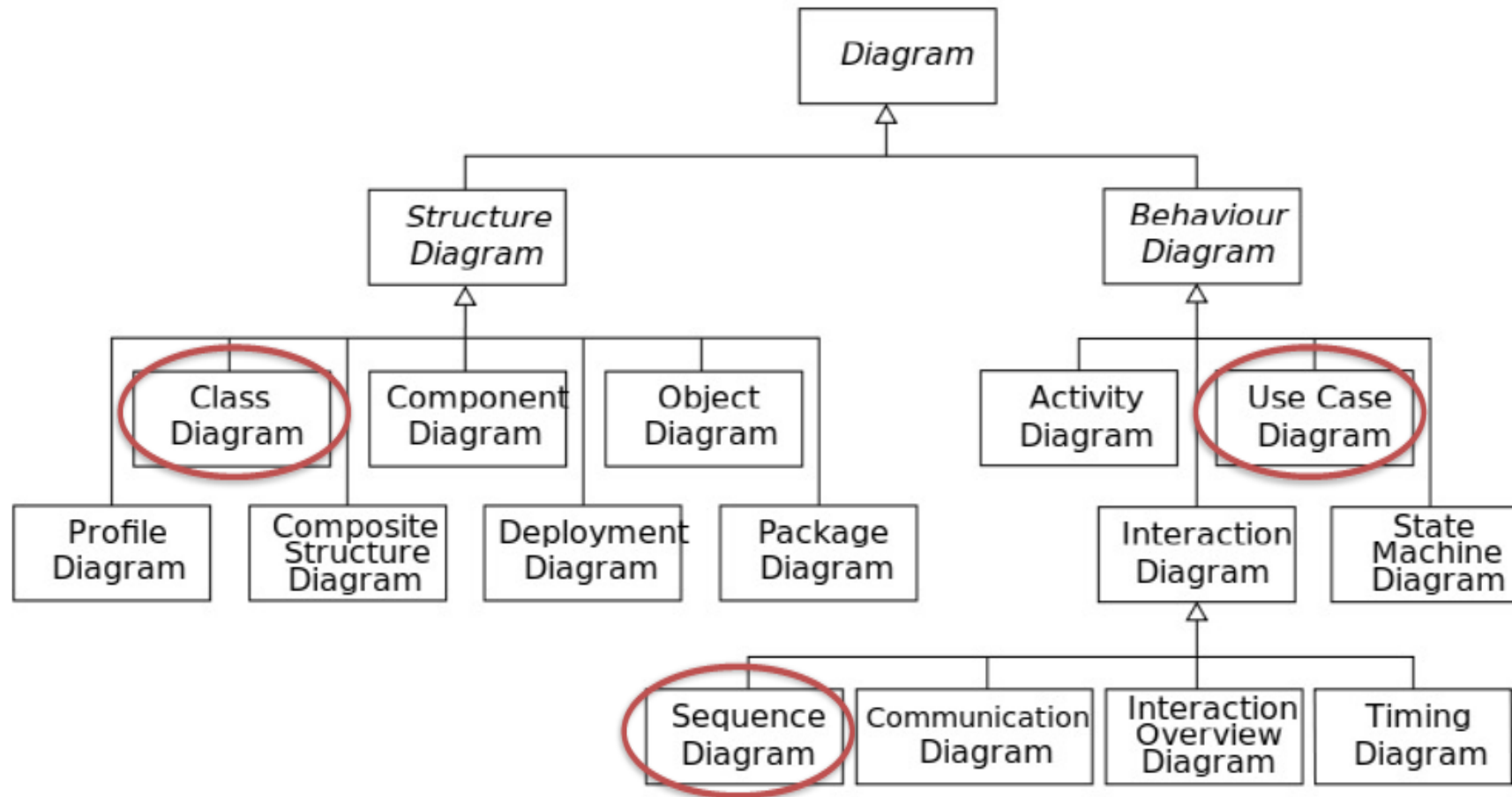
The **UML** is the standard language for

- visualizing,
- specifying,
- constructing,
- and documenting

the artifacts of a software intensive system

UML Diagrams

Types of UML Diagrams



UML Diagrams: Types

- **Use case diagrams,**
 - which show the interactions between a system and its environment.
- **Sequence diagrams,**
 - which show interactions between actors and the system
 - and between system components.
- **Class diagrams,**
 - which show the object classes in the system and the associations between these classes.
- **State diagrams,**
 - which show how the system reacts to internal and external events.

Interaction Models

Interaction models

- Use case diagrams and sequence diagrams may be used for interaction modelling.
- Modeling user interaction is important as it helps to identify user requirements.
- Modeling **system-to-system interaction** (Context):
 - highlights the communication problems that may arise.
- Modeling **component interaction** (Use/Seq) helps us:
 - understand if a proposed system structure is likely to deliver the required system performance and dependability

Use Case Modeling

- Use cases were developed originally to **support**
 - **requirements elicitation** and now incorporated into the UML.
- Each use case
 - represents a **discrete task** that involves **external interaction** with a system.
- Actors in a use case **may be people or other systems**.
- Represented diagrammatically to
 - provide an **overview** of the use case
 - and in a more detailed textual form.

Use Case Diagrams

- A **use case** is a particular **action** that a user needs to accomplish in a given system (such as making a deposit at an ATM machine).
- All the use cases provide the complete **external view** of a system.

Use Case Diagrams

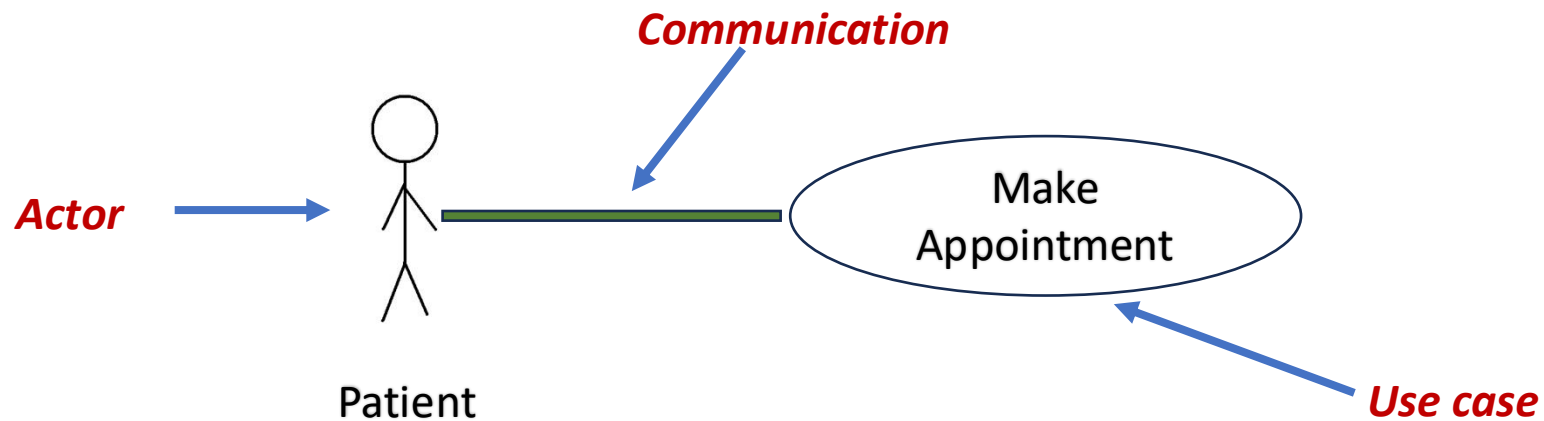
- Use case diagrams describe what a system does from the standpoint of an **external observer**
- Emphasis is on **what** a system does rather than how Use case diagrams are closely connected to **scenarios**
- **Scenario** is an example of what happens when someone interacts with the system

Use Case Diagrams

- A **use case** defines a **functional requirement** [of a system] which include interactions between the system and its actors.
- Use cases address the question of how actors **interact** with a system, and describe the **actions** the system performs

Use Case Diagrams

- A use case is a summary of scenarios for a single task or goal
- Consists of:
 - **Actor** is who or what initiates the events involved in that task
 - **Communication** associate is the connection between actor and use case

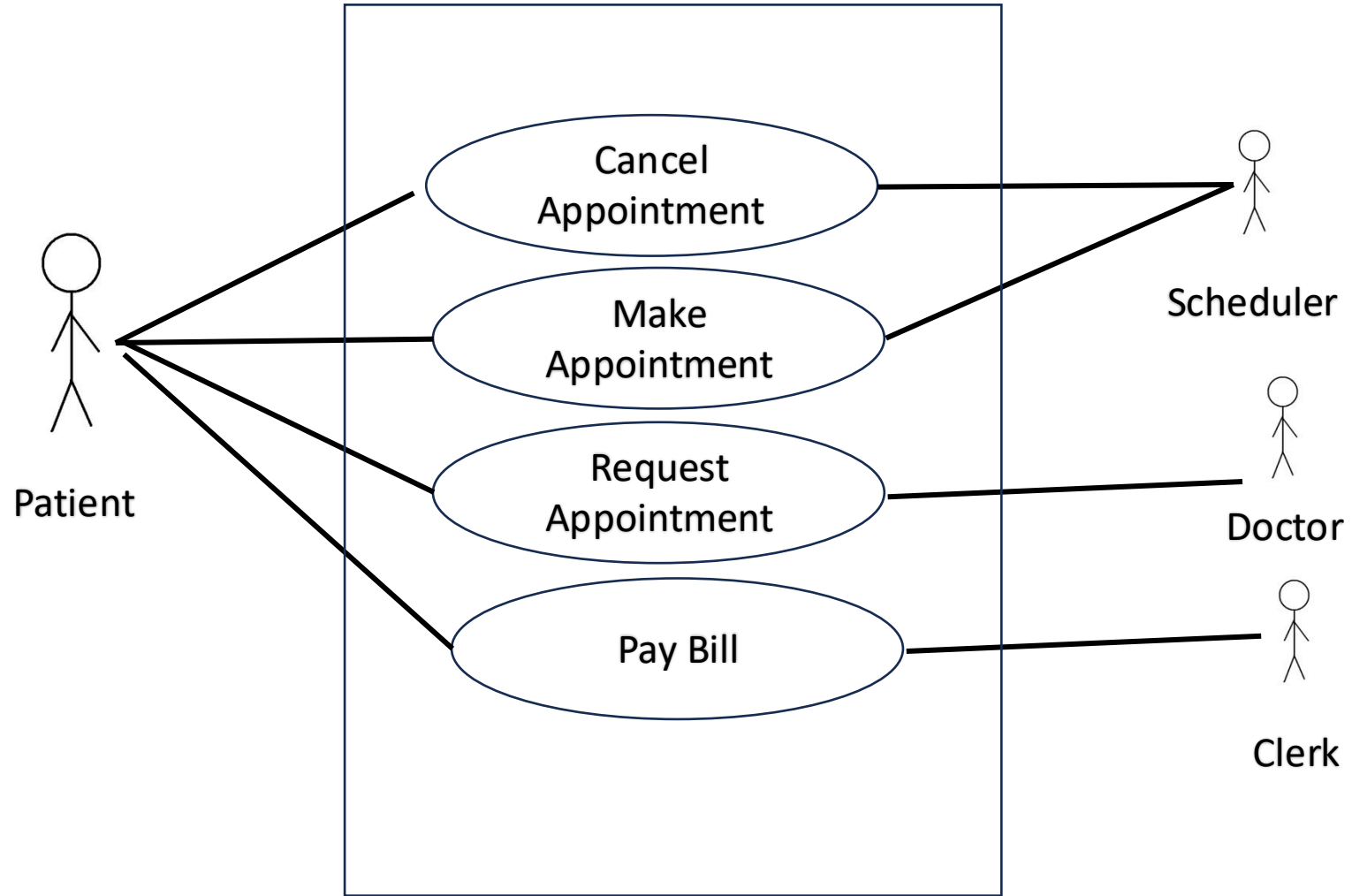


Actors in a Use Case Diagram

- The **actors** of a use case **are not necessarily individuals**.
- An **actor may be another system** interacting with the system being modeled.
- Always a stakeholder BUT all stakeholders don't have to be actors
i.e. product or system owner (stakeholders but not actors)
- User or outside system that **interacts** with system under design (your system) to produce something of value (intended result)

Use Case Example

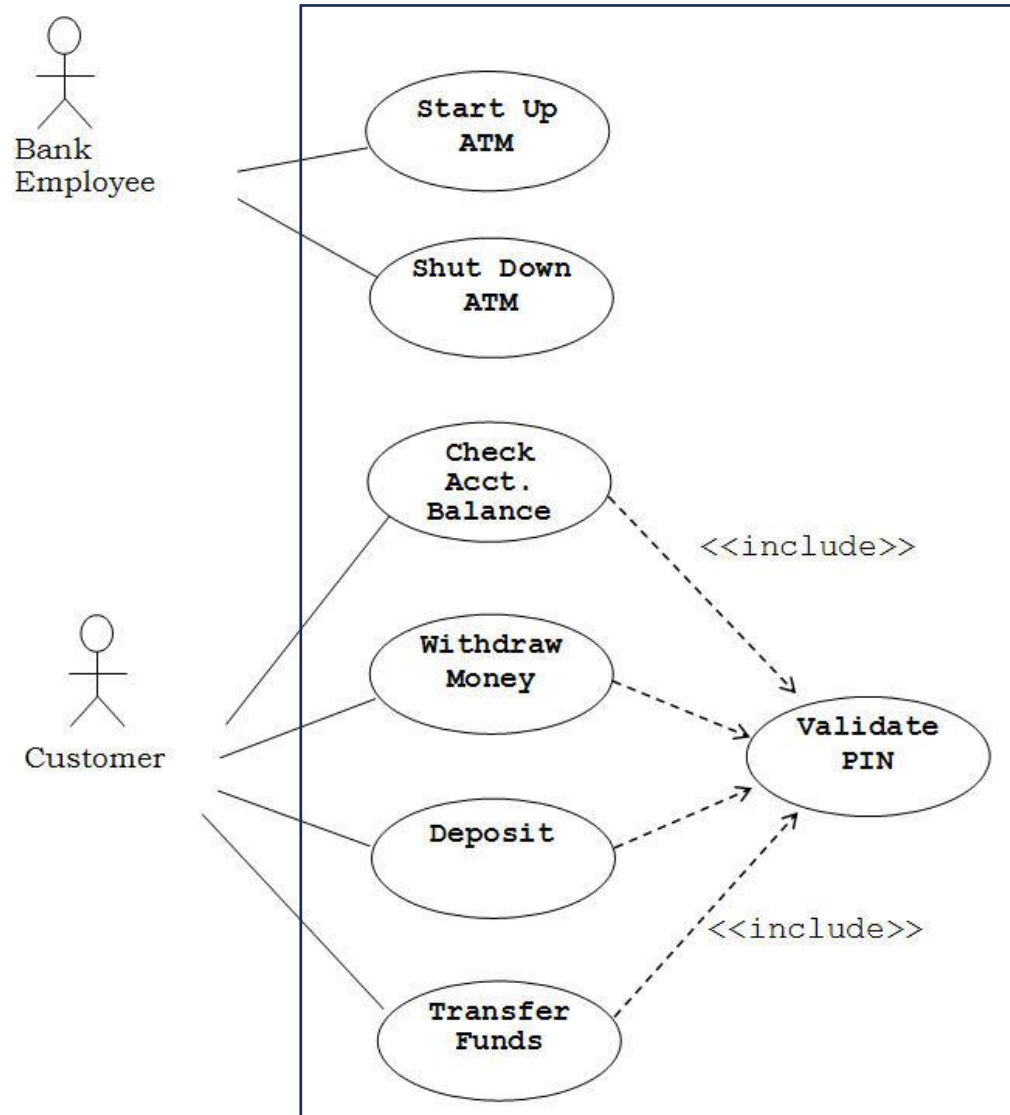
"A patient can call the clinic to make or cancel an appointment. The receptionist finds the nearest empty time slot to book and schedule the appointment or, otherwise cancel the appointment. The patient may also call a clerk to request medicine and pay their bill. When requesting medicine a doctor must check the request against a patient's record."



Use Case -- Include

- A **use case include** is used to **factor out steps** that occur in more than one scenario.
- The include is indicated by a **dashed line** from a use case to the included steps, with an **open arrowhead** at the included set of steps.

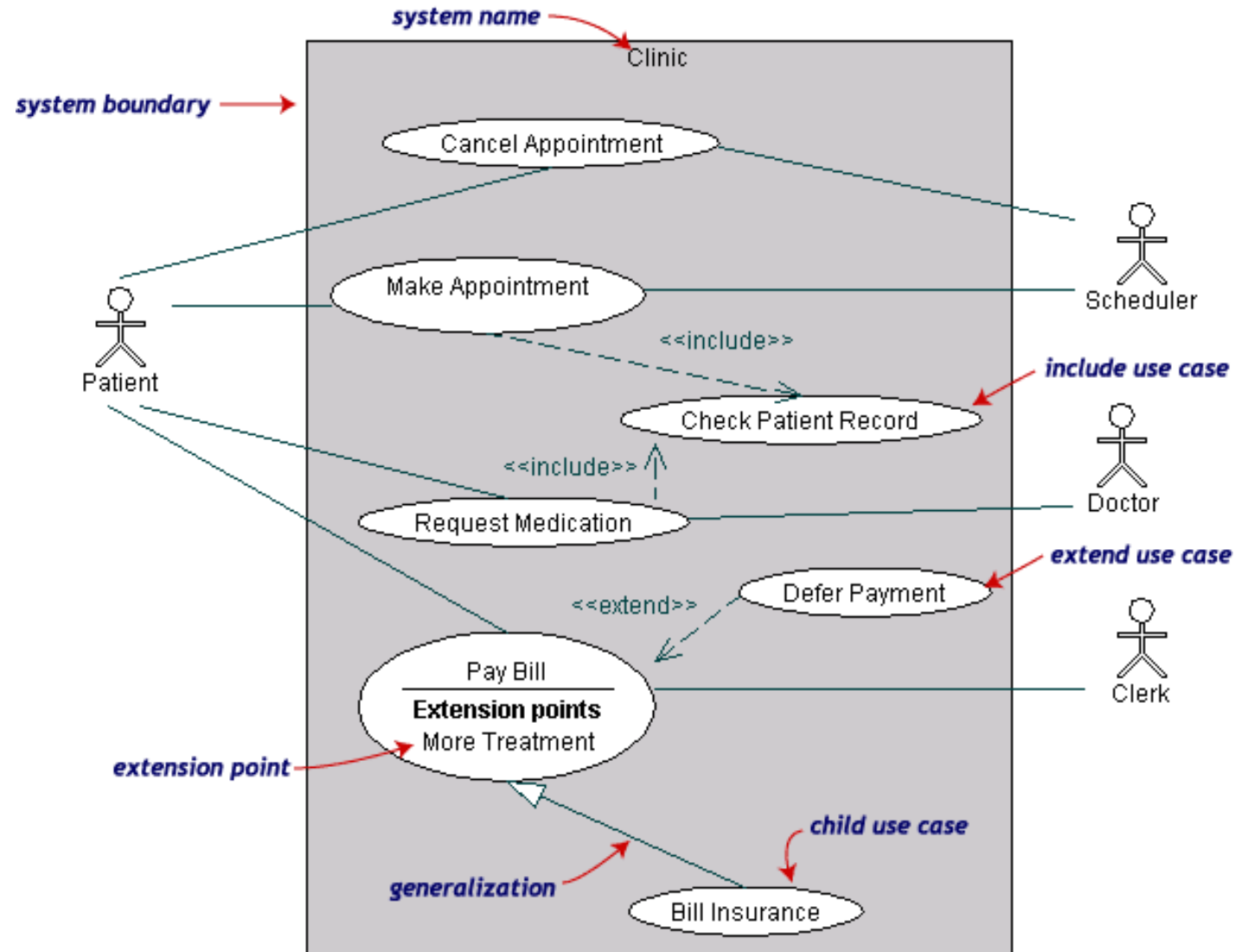
Example Use Case Diagram for an ATM Machine



- Note that validate pin is not a use case. (Validating their pin is not a goal of the user, but just a necessary set of steps.)

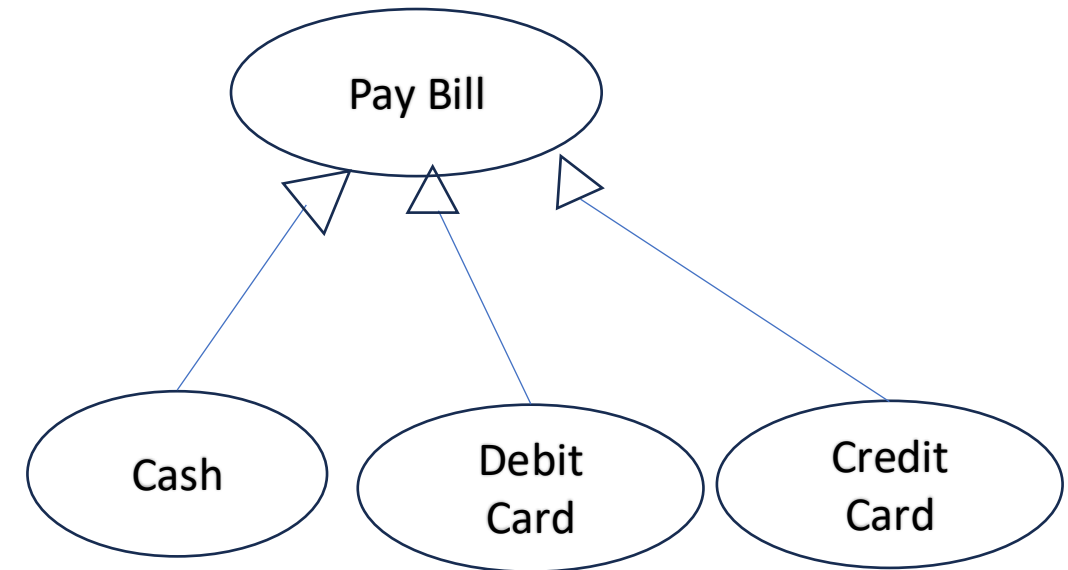
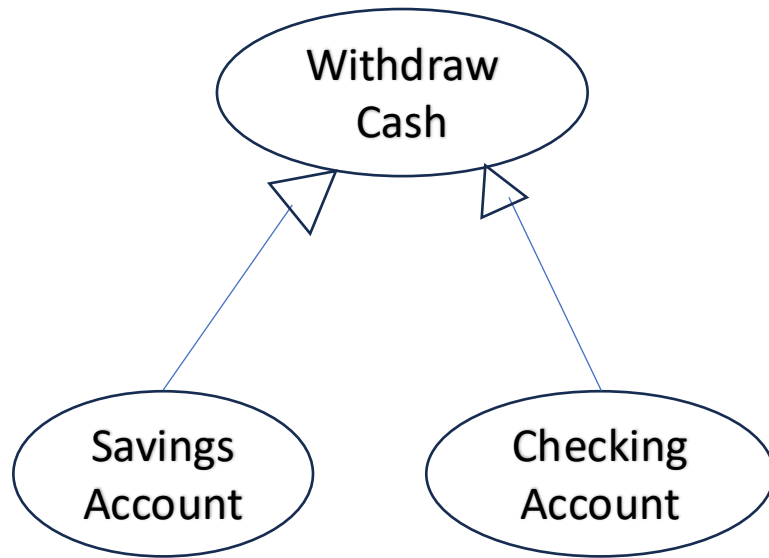
System Boundary

- A system boundary rectangle separates the system from the external actors.



Use Case Generalization

- A **use case generalization** is a way to indicate that a given use case is a variation of, or specific version of, another use case (called the **base use case**).
- To denote this, a line is given from a use case to its base use case, with a **closed unfilled arrowhead** at the base use case.

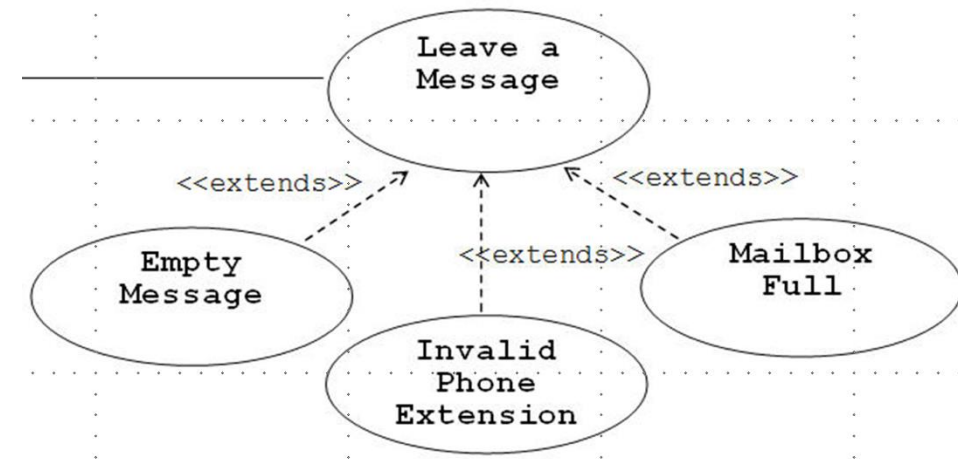


Use Case Extension

“A **use case extension** from one use case (called the extension use case) to another use case (called the base use case) indicates that the extension use case will **extend** (or be inserted to) and **augment** the base use case.

A use case may extend multiple use cases, and a use case may be extended by multiple use cases.”

- from “Learning UML” by Sinan Si Alhir



Use Case Extension

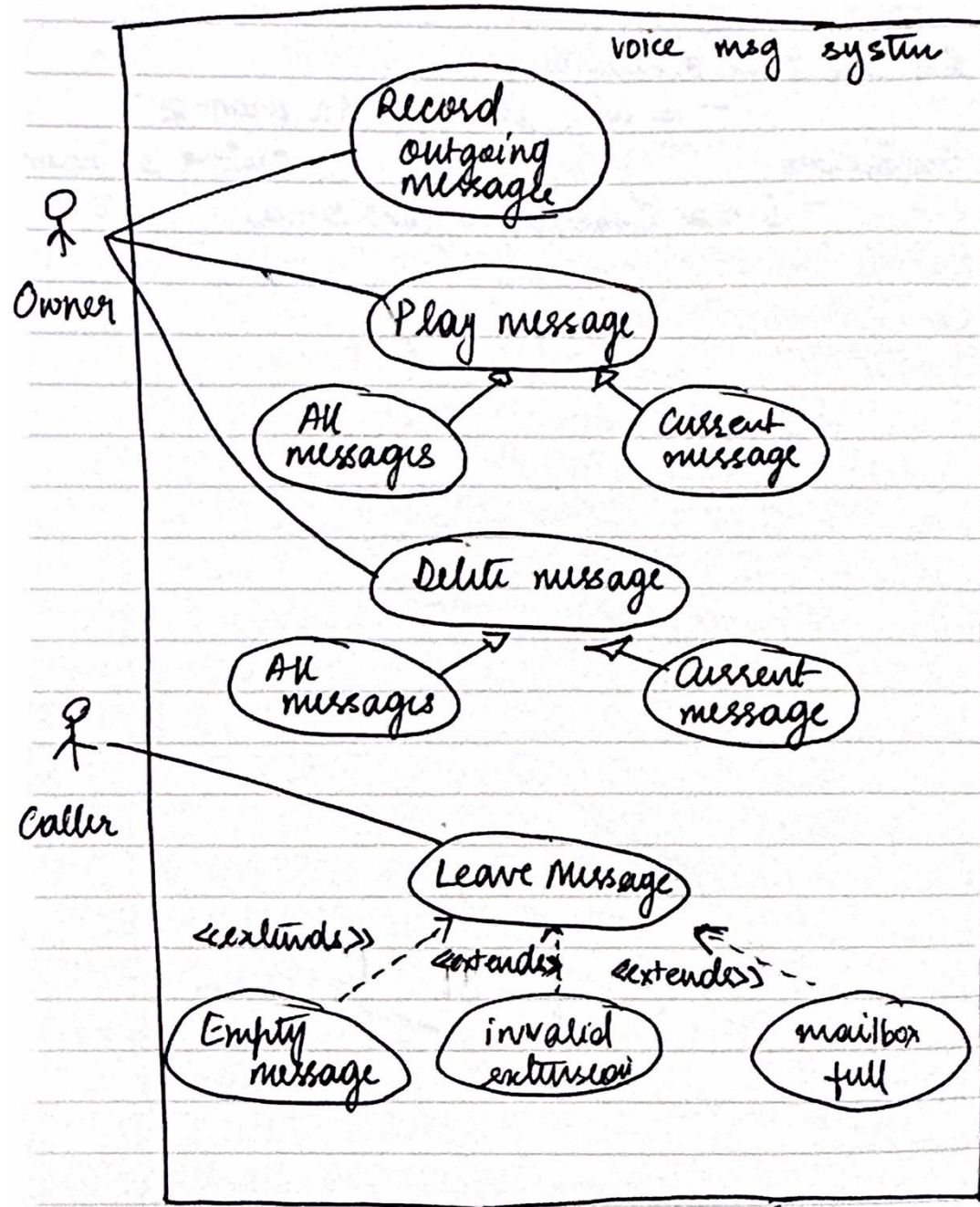
- In use case extension, extension points indicate the only ways that a given use case may be modified.
- Extension points, determine when the extended case is appropriate, are written inside the base case.

Use Case Extension

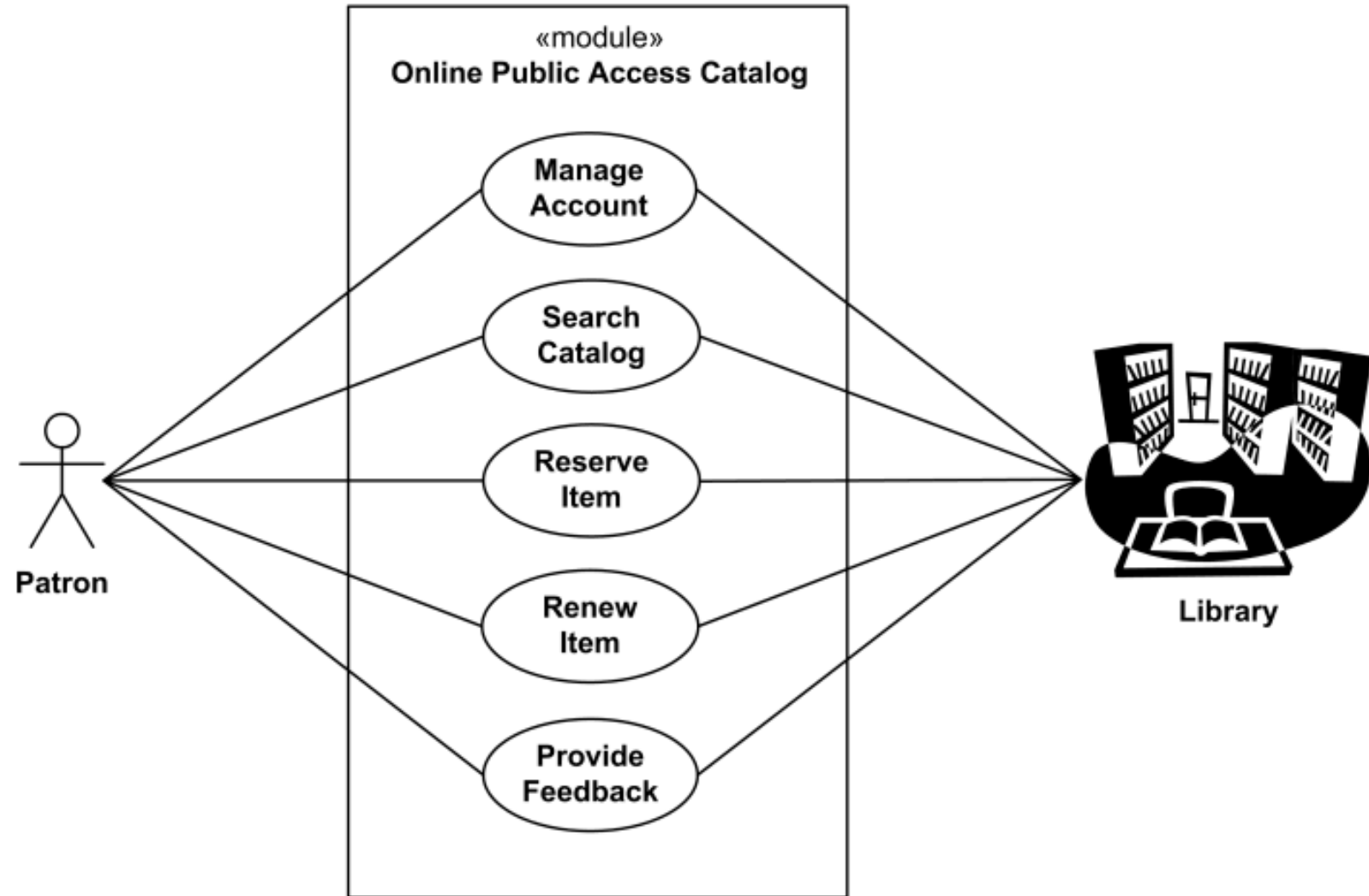
- For example, in a use case that indicates the steps for a customer making an online purchase, there are considerations such as,
 - what to do if a credit card is rejected?
 - what to do if the product is no longer available?
 - what to do is the using credit from a prior refund?etc.

Each of these could be a generalization of a Make Purchase use case in a given system.

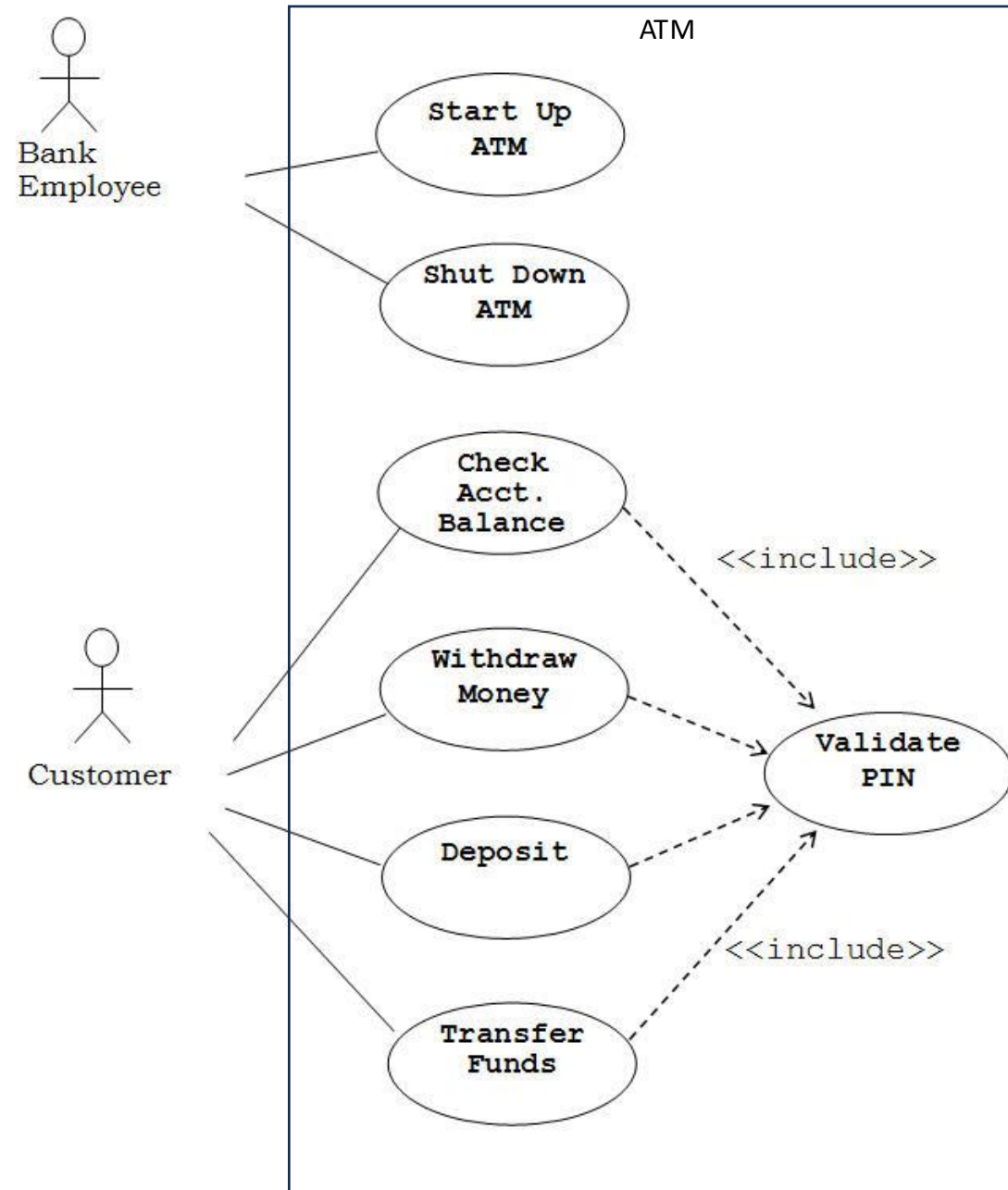
Example Use Case Diagram for a Voice Mail System



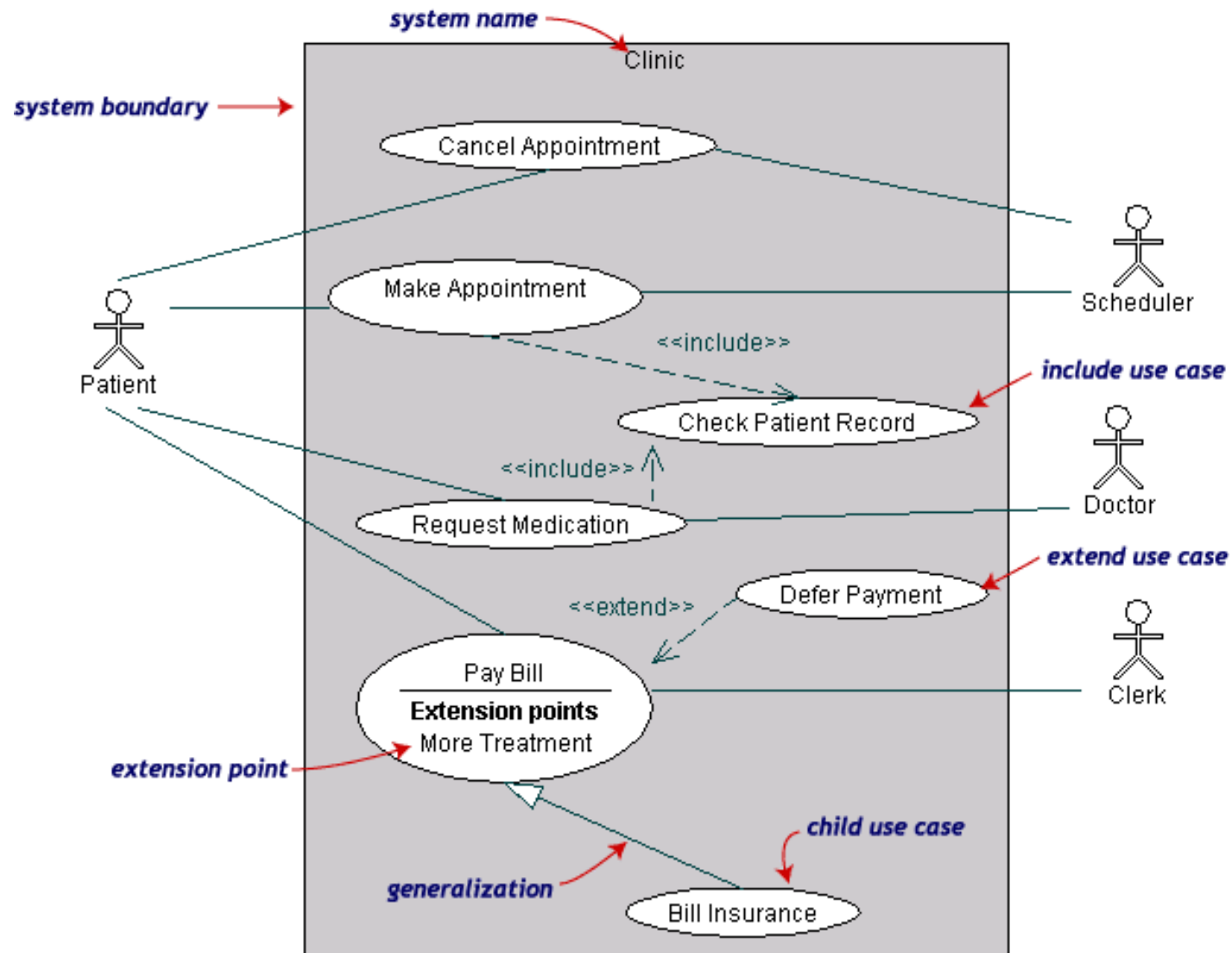
Use Case Diagram for a Library



Example Use Case Diagram for an ATM Machine

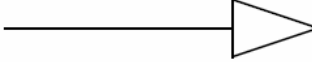


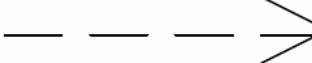
Use Case Example

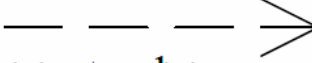


Summary

- What is system boundary?
- Generalization?
- Include?
- Extend?

Generalization 

Include 
<<include>>

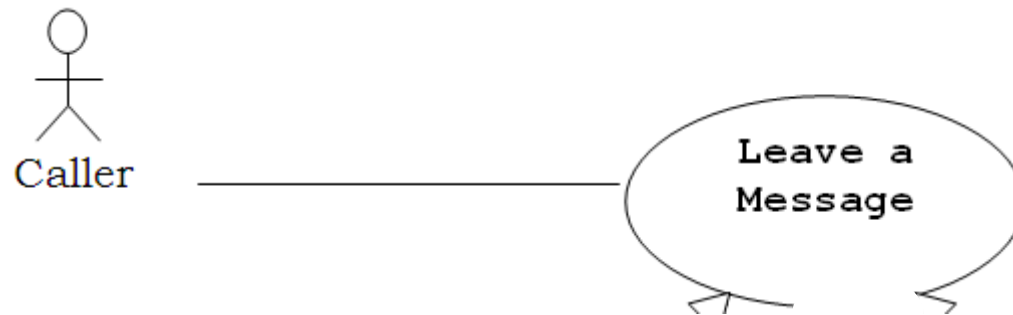
Extend 
<<extend>>

Scenarios

- Scenarios
 - real-life examples of what system does or how it can be used
 - **sequence of steps** describing interaction between user and system
- Should include:
 - Description of **starting** situation
 - Description of normal **flow** of events
 - Description of **what can go wrong**
 - Information about other **concurrent** activities
 - Description of state when scenario **finishes**

Use Case of a Voice Mail System

What is the scenario?



Example Scenario (for a voice mail system)

Leave a Message

1. The caller dials the main number of the voice mail system.
2. The voice mail system speaks a prompt.
“Enter mailbox number followed by #.”
3. The user types in the extension number of the message recipient.
4. The voice mail system speaks.
“You have reached mailbox xxxx. Please leave a message now.”
5. The caller speaks the message.
6. The caller hangs up.
7. The voice mail system places the recorded message in the recipient's mailbox.

Use Case Examples

Use Case 2 Get Paid for Car Accident

Primary Actor: Claimant

Scope: Insurance company ("MyInsCo")

Level: Summary

Stakeholders and Interests:

Claimant—to get paid the most possible.

MyInsCo—to pay the smallest appropriate amount.

Department of Insurance—to see that all guidelines are followed.

Precondition: None.

Minimal Guarantees: MyInsCo logs the claim and all activities.

Success Guarantees: Claimant and MyInsCo agree on amount to be paid; claimant gets paid that.

Trigger: Claimant submits a claim.

Main Success Scenario:

1. Claimant submits claim with substantiating data.
2. Insurance company verifies claimant owns a valid policy.
3. Insurance company assigns agent to examine case.
4. Insurance company verifies all details are within policy guidelines.
5. Insurance company pays claimant and closes file.

Extensions:

- 1a. Submitted data is incomplete:
 - 1a1. Insurance company requests missing information.
 - 1a2. Claimant supplies missing information.
- 2a. Claimant does not own a valid policy:
 - 2a1. Insurance company denies claim, notifies claimant, records all this, terminates proceedings.
- 3a. No agents are available at this time.
 - 3a1. (What does the insurance company do here?)
- 4a. Accident violates basic policy guidelines:
 - 4a1. Insurance company denies claim, notifies claimant, records all this, terminates proceedings.
- 4b. Accident violates some minor policy guidelines:
 - 4b1. Insurance company begins negotiation with claimant as to amount of payment to be made.

- Actor (user) is claimant who needs to be reimbursed for accident
- Main flow of events
 - Main Success Scenario
 - What happens in normal situation if things go as expected
- Exceptions to the rule
 - Extensions
 - What happens under less-than-ideal condition(s)

Use Case Examples

Name	Save item for purchase.
ID	UC_001
Description	While browsing items in the eStore, a user finds an item he is not ready to purchase yet, but he wants to save it to a list so that he can later find the item that he was previously interested in.
Actors	eStore customer.
Organizational Benefits	Increase sales by helping the customer remember products he was previously interested in.
Frequency of Use	20% of users save an item to be bought later each time they visit the site. 50% of saved items are purchased within one year of the saved date.
Triggers	The user selects an option to save an item.
Preconditions	User is viewing an item in the catalog.
Postconditions	The item selected to be saved is visible to the user when he views his saved items. The item selected to be saved is reflected as a saved item when the user views his eStore search and browse results.
Main Course	1. System prompts user to confirm saving selected item instead of purchasing it right away. 2. User confirms to save now (see EX1). 3. System determines user is not logged in and redirects user to log on (see AC1). 4. User logs on (see AC2, AC3). 5. System stores the saved item (see EX2). 6. System redirects the user to their saved items list to view the full list.
Alternate Courses	AC1 System determines user is already logged on. 1. Return to Main Course step 5. AC2 User logs off again. 1. Return user to Main Course step 3. AC3 User does not have an account already. 1. User creates an account. 2. System confirms account creation. 3. Return user to Main Course step 4.
Exceptions	EX1 User decides to purchase the item now. 1. See "Purchase item" Use Case. EX2 System fails on saving item to list. 1. System notifies user that an error has occurred. 2. Return user to Main Course step 1.

- Actor is user browsing online store who wants to save an item to be purchased later
- Normal Condition (if)
 - Main Course
- Alternates (else if {} else if {}....)
 - Possible alternatives to main course
- Exceptions
 - Actions that occur which do not fall under normal course path or alternative course paths

Class Activity – Use Case Diagram

- Given below Actors and Use Cases, please draw the use case diagram for an airport/airline transportation support system.

Actors

- passenger,
- check-in representative,
- TSA officer,
- Baggage management system

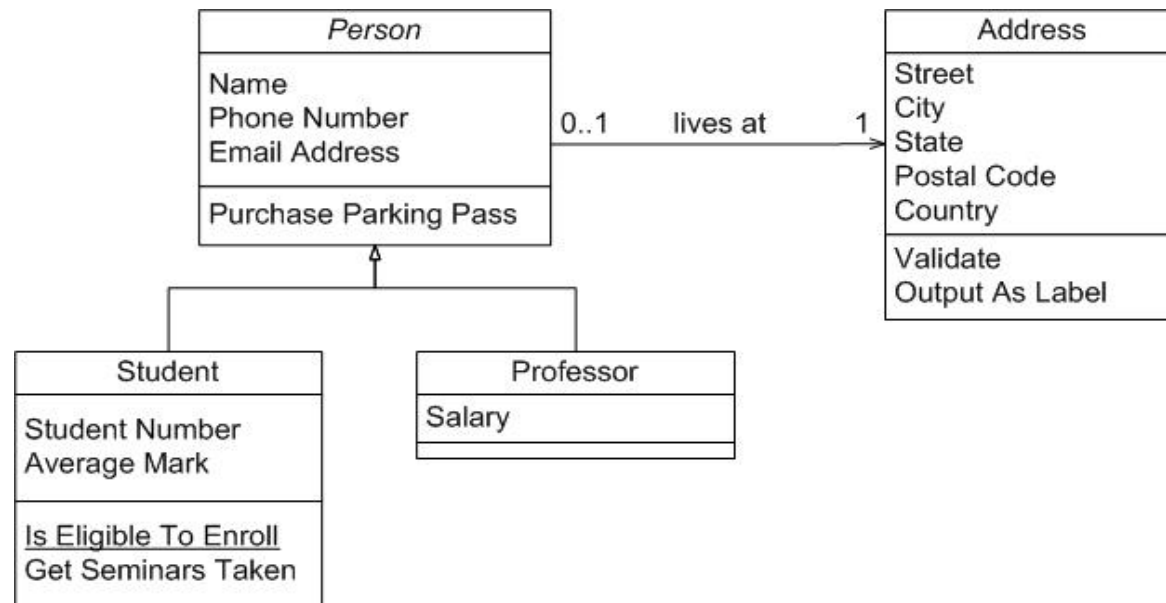
Use Cases

- Check-in at terminal, mobile check-in, online check-in
- Check baggage
- Issue boarding pass, boarding
- Change flight
- Security check

Structural Models – Class diagrams

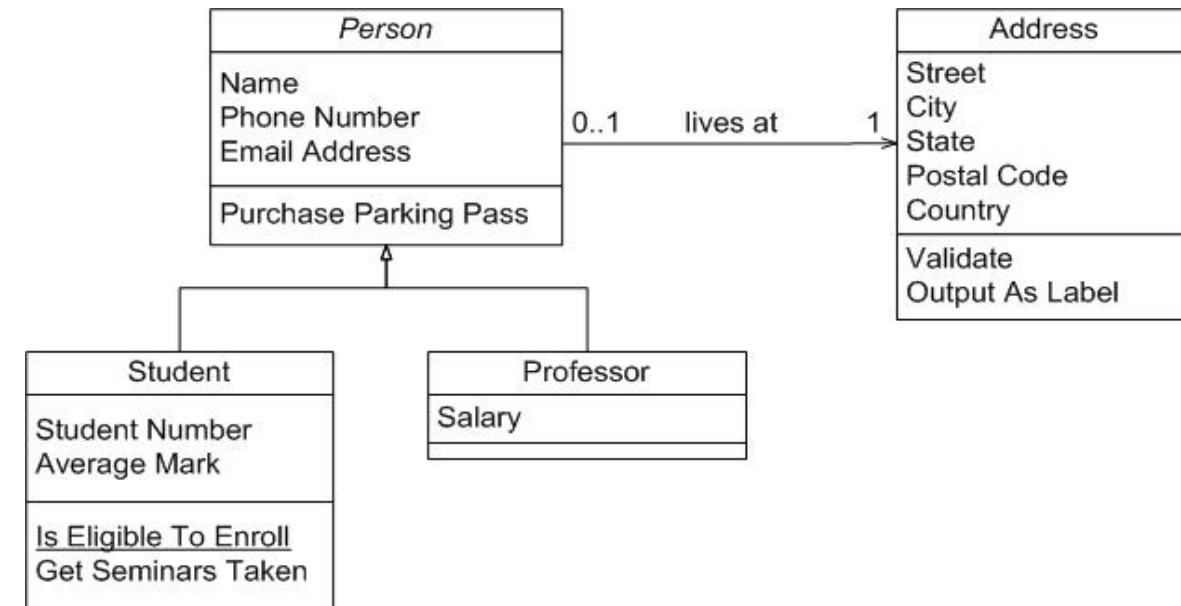
Structural models

- Class diagrams show static structure
 - This is the diagram that is used to generate code



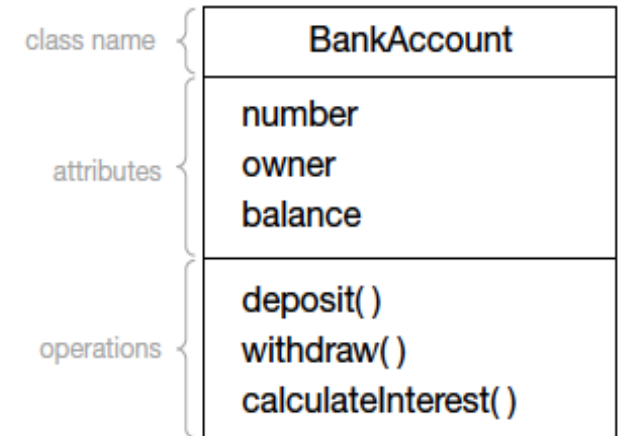
Class Diagrams

- Class diagrams are used:
 - during requirements analysis to model application domain concepts
 - during system design to model subsystems
 - during object design to specify the detailed behavior and attributes of classes.



Classes

- A class defines a set of objects with
 - common structure,
 - common behavior,
 - common relationships
 - and common semantics
- A class is drawn as a rectangle with three compartments
 - Naming standards should be created
 - e.g., all classes are singular nouns starting with a capital letter



Where to Find classes?

- Finding classes using **noun/verb analysis**
- Suitable sources of information are:
 - the Supplementary Requirements Specification (if it exists);
 - use cases;
 - the project Glossary;
 - anything else (architecture, vision documents, etc.).

How to Find classes?

- Finding classes using noun/verb analysis
 - nouns – e.g. flight; Class
 - noun phrases – e.g. flight number; Attribute
 - verbs – e.g. allocate; Operation/Responsibility
 - verb phrases – e.g. verify credit card. Operation/Responsibility

How to Find classes?

- Finding classes using CRC analysis
 - CRC analysis – this stands for class, responsibilities, and collaborators

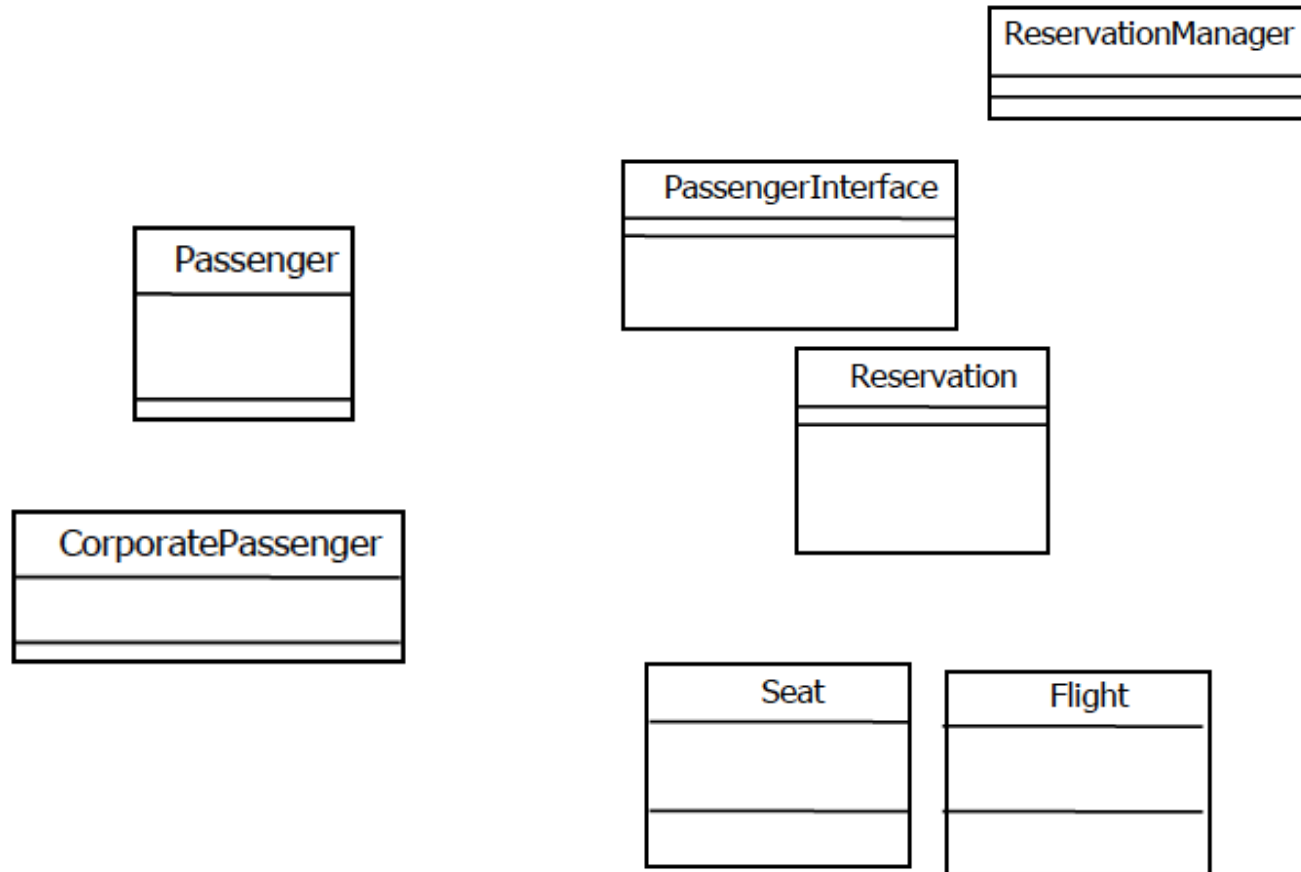
Class name: BankAccount	
Responsibilities: Maintain balance	Collaborators: Bank



Collaborators are other classes that may collaborate with this class to realize a piece of system functionality.

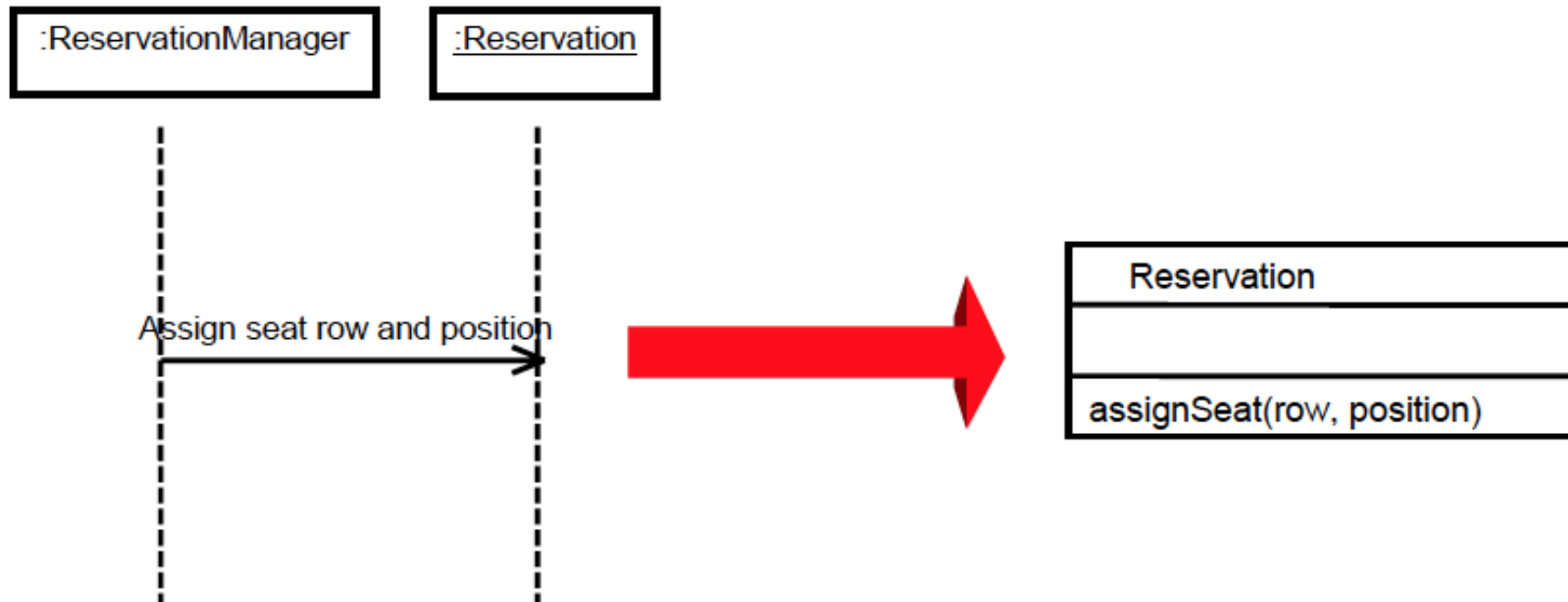
Classes: Names

- The class **name** is the only mandatory information



Classes: Operations

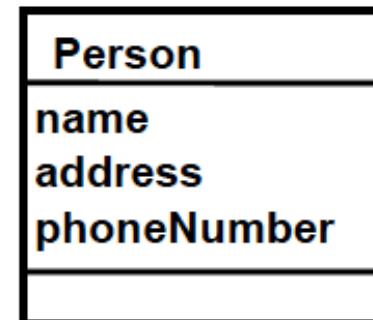
- The **behavior** of a class is represented by its operations
 - **Operations** may be found by examining **interaction diagrams**



Classes: Attributes

- The structure of a class is represented by its **attributes**
- Attributes may be found by examining class definitions, the problem requirements, business rules and by applying domain knowledge

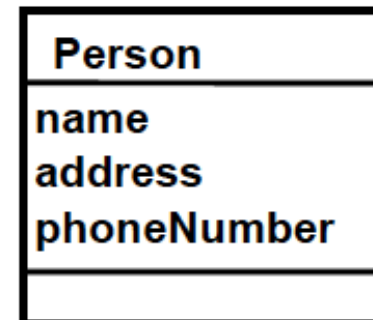
The name, address and phone number for each person is needed before a reservation can be made



Classes: Attributes

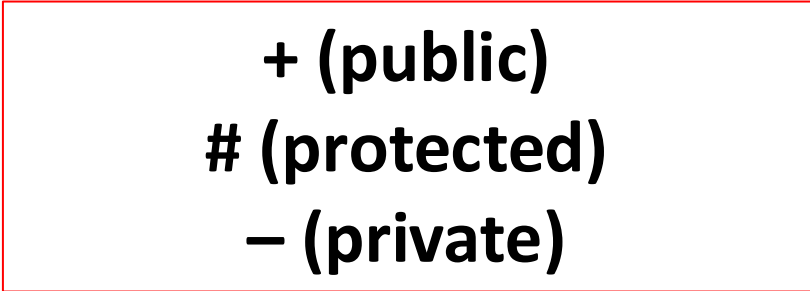
- The structure of a class is represented by its **attributes**
- Attributes may be found by examining class definitions, the problem requirements, business rules and by applying domain knowledge

The name, address and phone number for each person is needed before a reservation can be made



Visibility of Class Members

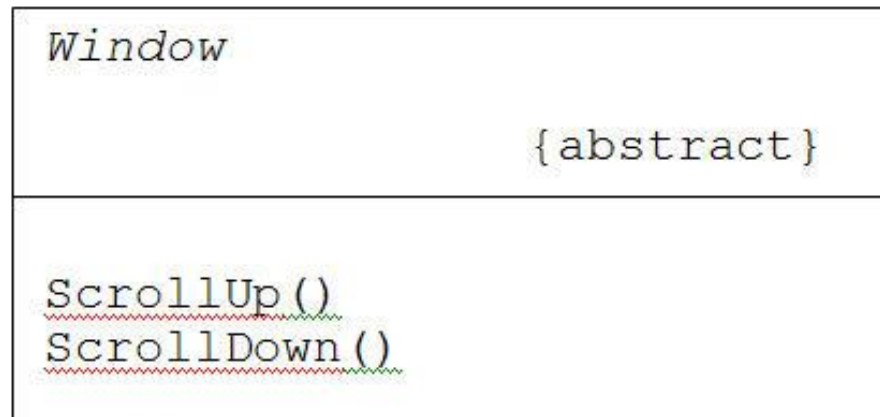
- Classes can have variables and methods with different levels of access
- In object-oriented programming we have three visibilities or access levels
 - Public
 - Private
 - Protected
- Following is the UML representation of these access specifiers



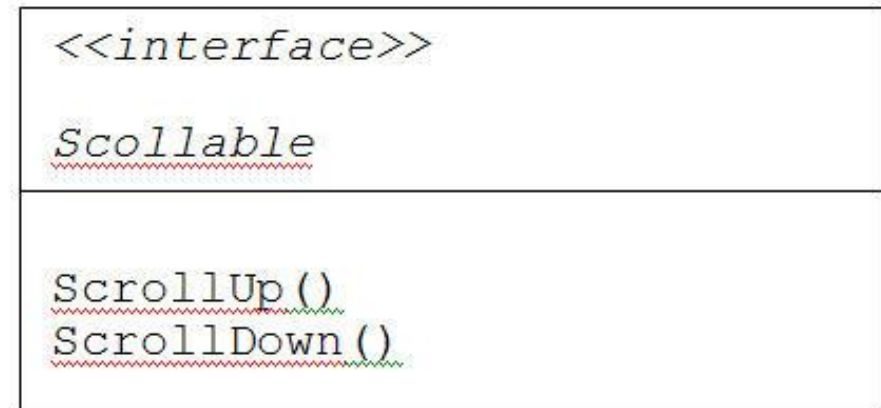
+ (public)
(protected)
– (private)

Abstract Class and Interface

Abstract Class Diagrams



Interface Diagrams



Relationships in Class Diagrams

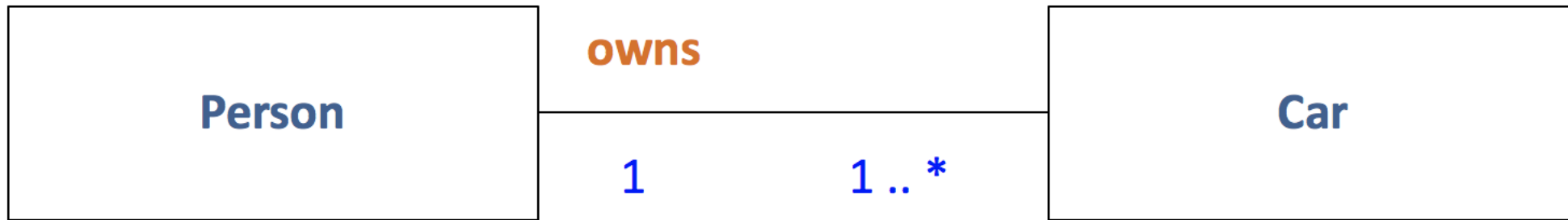
- Association
- Aggregation
- Composition
- Generalization (subtype)
- Interface Implementation

Association

- An association represents a relationship between instances of classes (i.e., objects).
 - Each such association has two association ends.
- An association indicates that one object calls methods of the other (and thus has access to it).
- An association end ***may have a role name*** which makes clear what the relationship is when not otherwise obvious.
- Associations are the most common type of relationship in class diagrams.

Example

Associations are denoted by solid lines

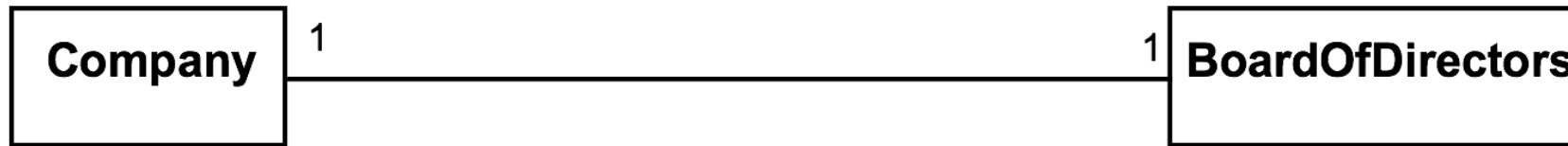


owns - role name

1..* - multiplicity

Multiplicity

- One-to-one
 - For each company, there is exactly one board of directors
 - A board is the board of only one company
 - A company must always have a board
 - A board must always be of some company

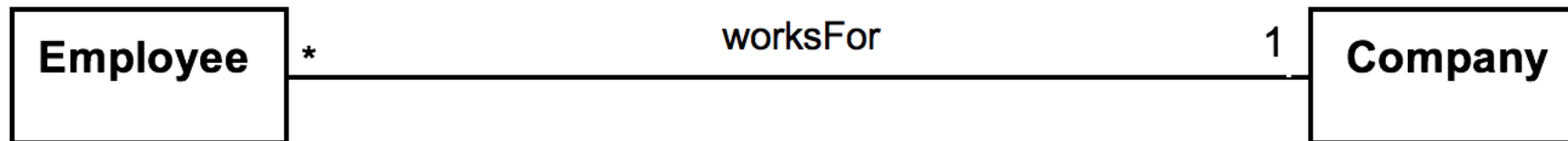


Multiplicity

- Many-to-one

- A company has many employees,
- An employee can only work for one company.
- A company can have zero employees
 - E.g. a 'shell' company
- Is it possible to be an employee without working for a company?

*	⇒ 0, 1, or more
1	⇒ 1 exactly
2..4	⇒ between 2 and 4, inclusive
3..*	⇒ 3 or more (also written as "3..")

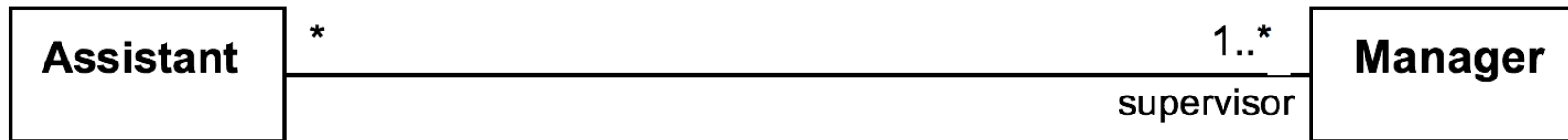


Multiplicity

- Many-to-many

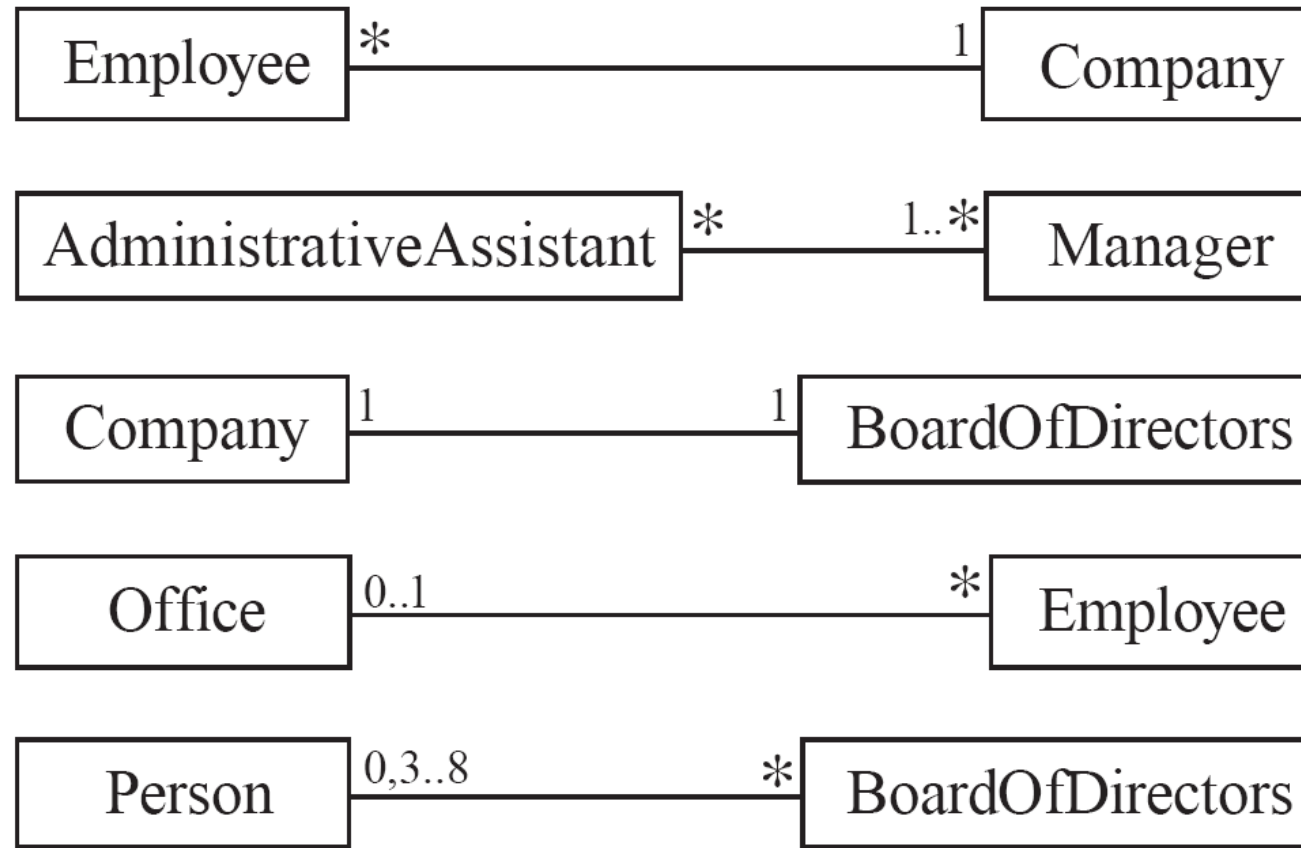
- An assistant can work for many managers
- A manager can have many assistants
- Assistants can work in pools
- Managers can have a group of assistants
- Some managers might have zero assistants.
- Is it possible for an assistant to have, perhaps temporarily, zero managers?

*	⇒ 0, 1, or more
1	⇒ 1 exactly
2..4	⇒ between 2 and 4, inclusive
3..*	⇒ 3 or more (also written as "3..")



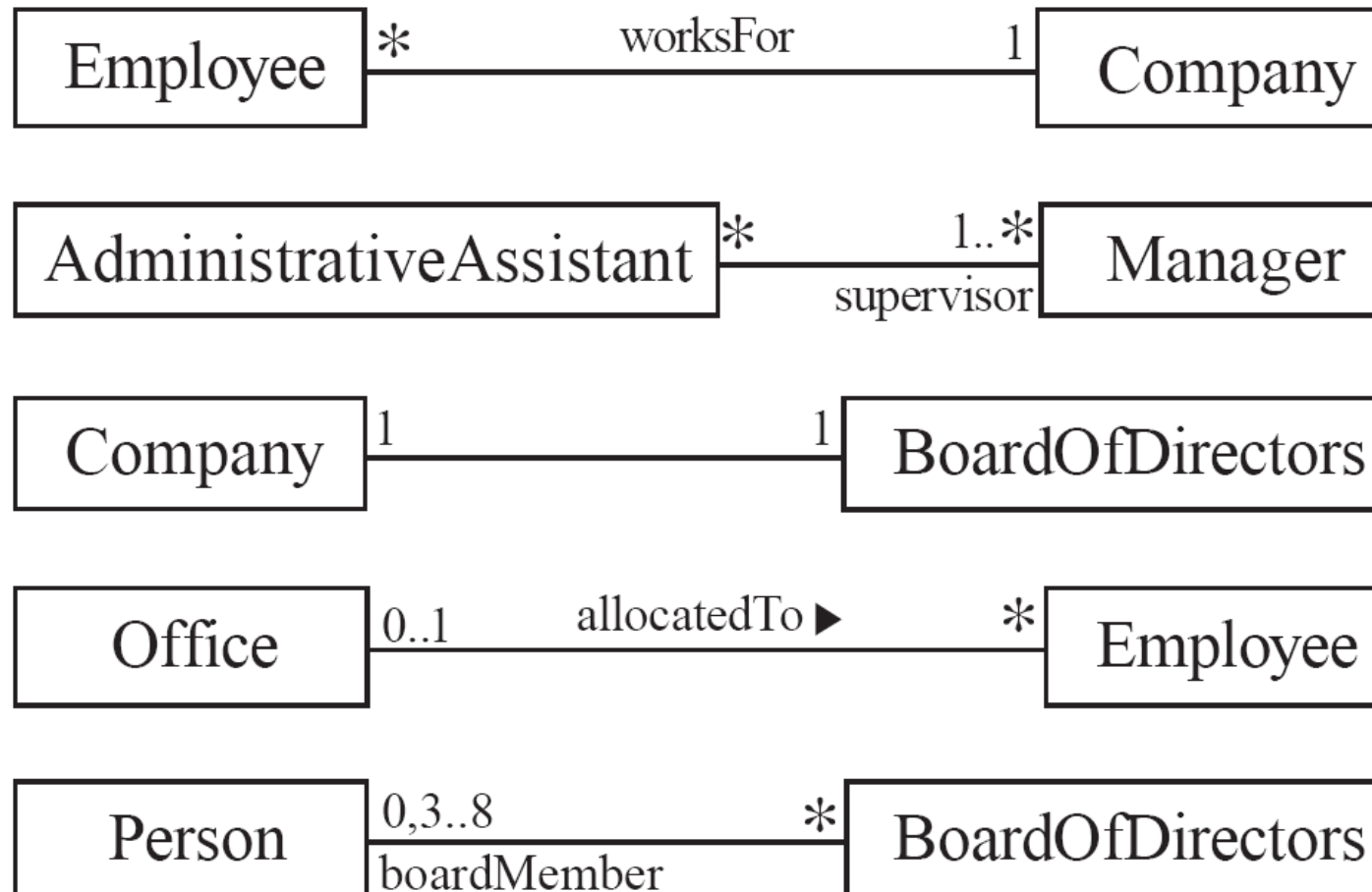
Associations and Multiplicity

- Symbols indicating multiplicity are shown at each end of the association



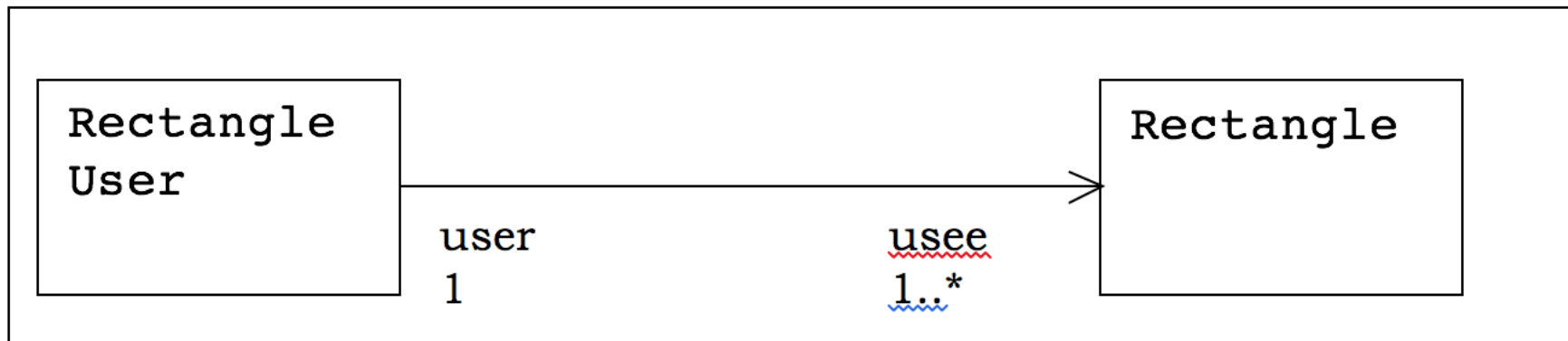
Labelling Associations

- Each association can be labelled, to make explicit the nature of the association



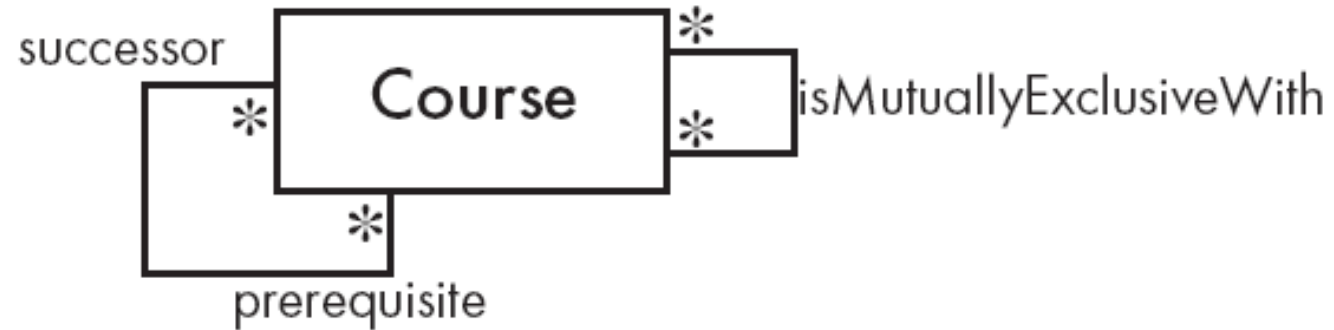
Navigation

- Navigation arrow only used when the direction of method calls is not obvious.
- Never use arrow on both ends.
 - If objects call each other, then just use line with no arrows.
- If no navigation arrow is given, then the association is considered bi-directional.
- Here, the navigation arrow implies that the Rectangle User will makes requests of the Rectangle, but the Rectangle will not make requests of the Rectangle User.

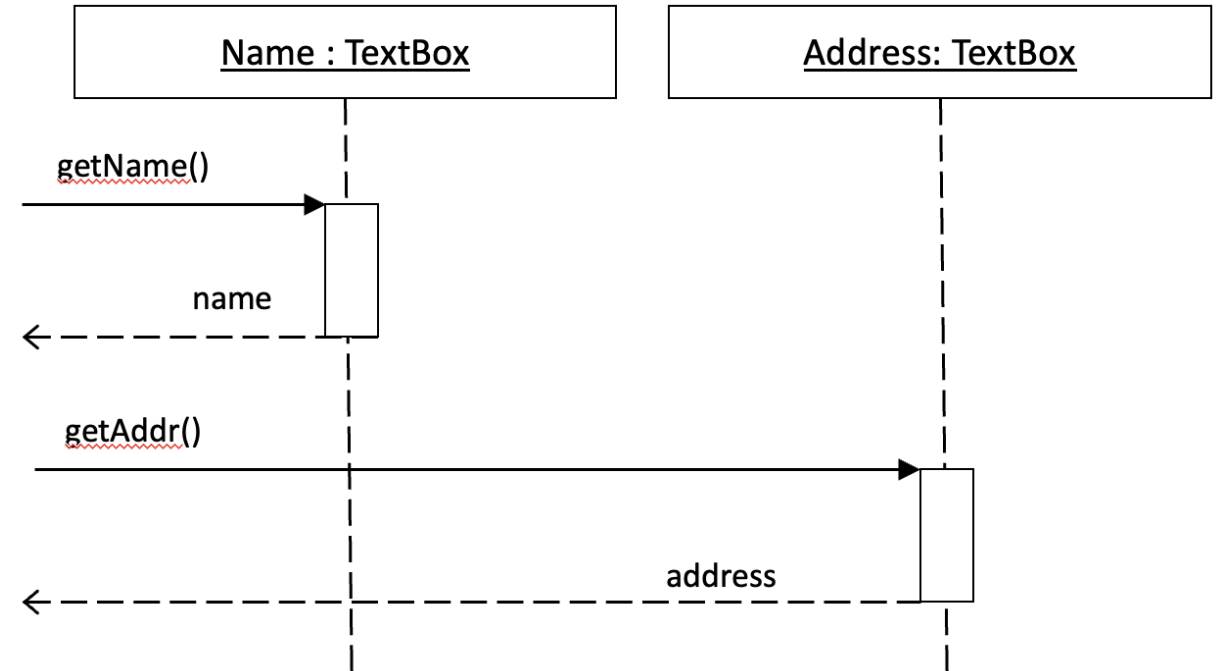
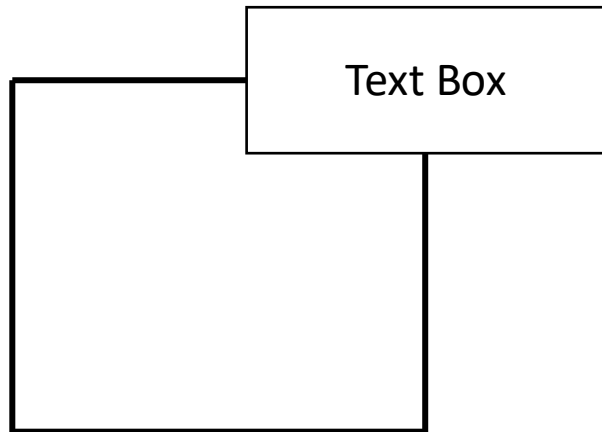


Reflexive Associations

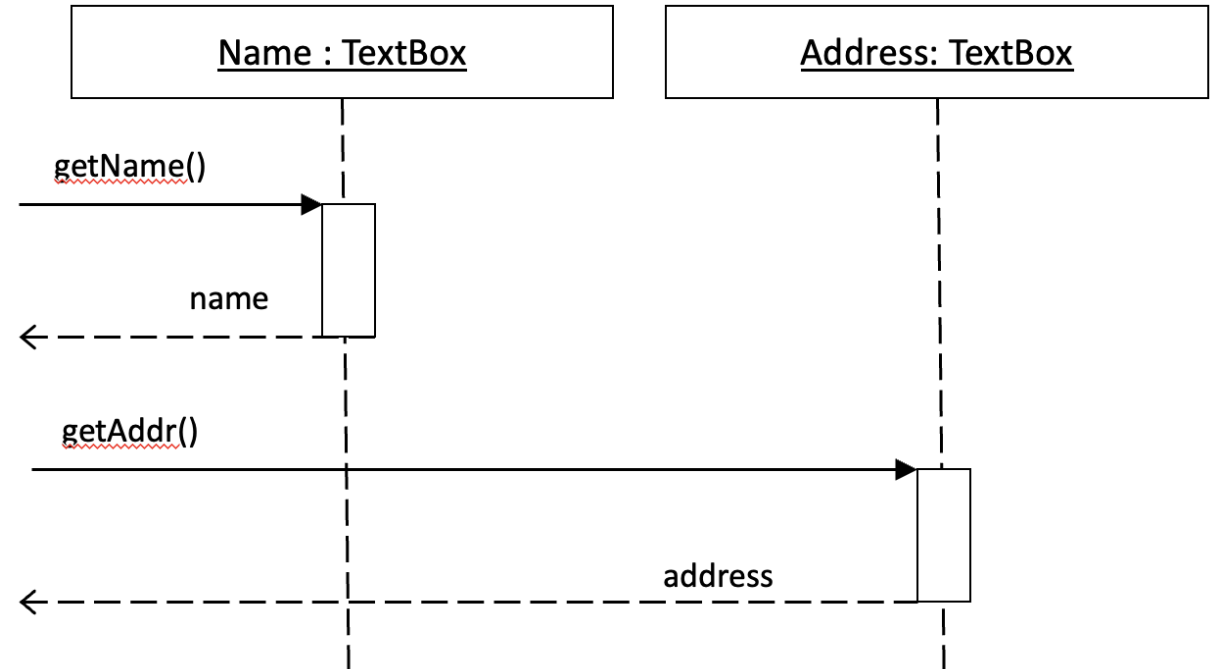
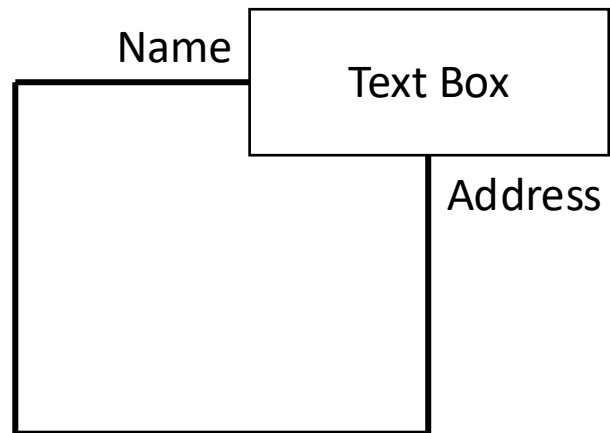
- It is possible for an association to connect a class to itself



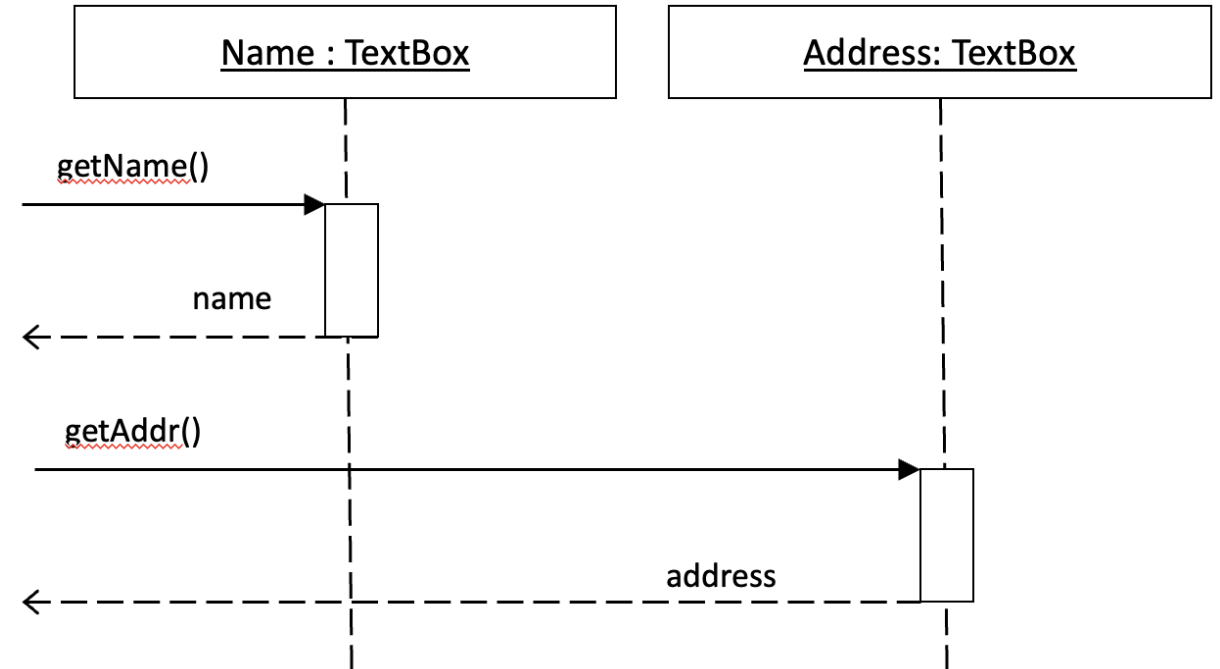
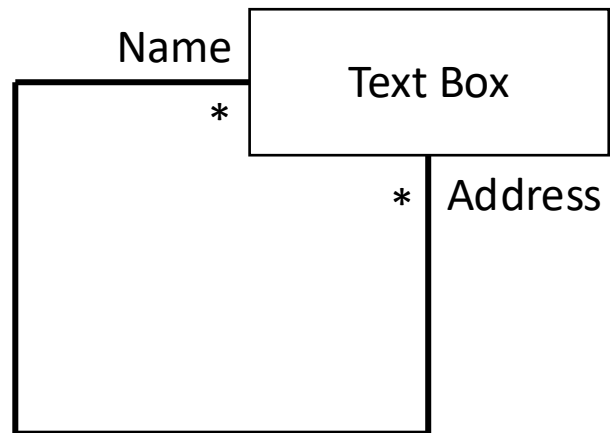
Reflexive Associations



Reflexive Associations

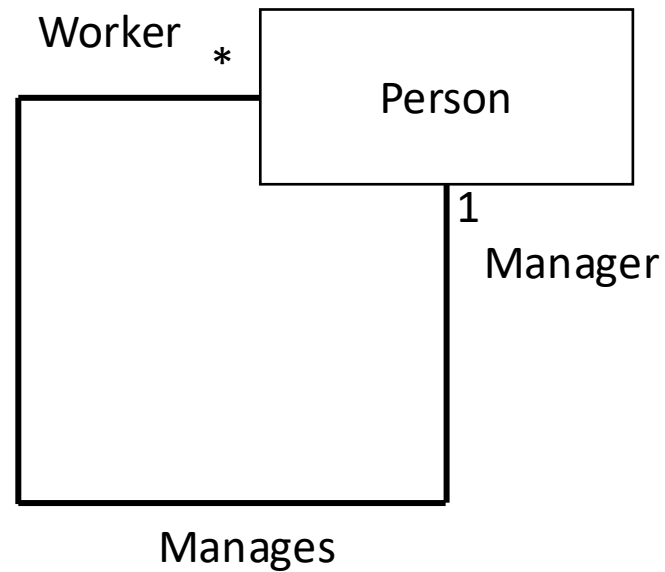


Reflexive Associations



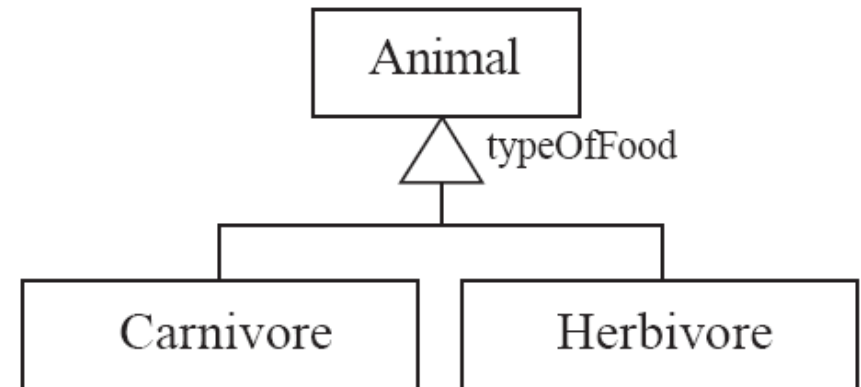
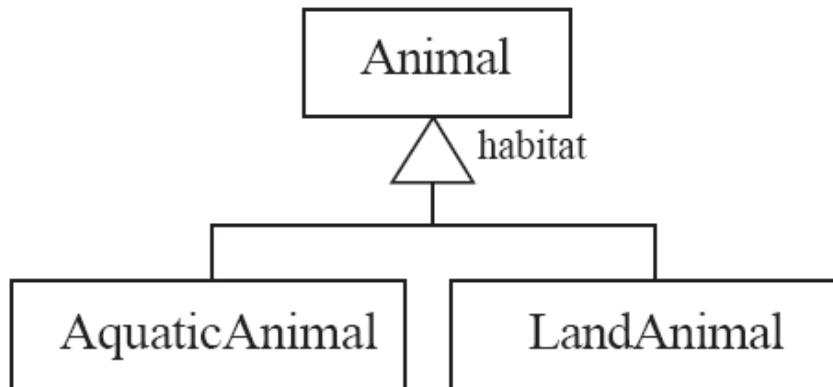
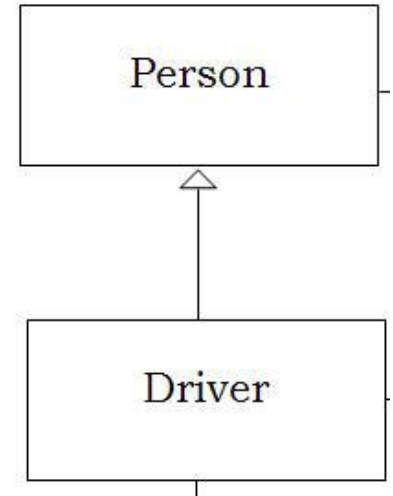
Class Activity

- Draw a class diagram representing the relationship between manager and worker. Take into account that a person can have both a manager and a worker. Annotate associations with roles and multiplicities.

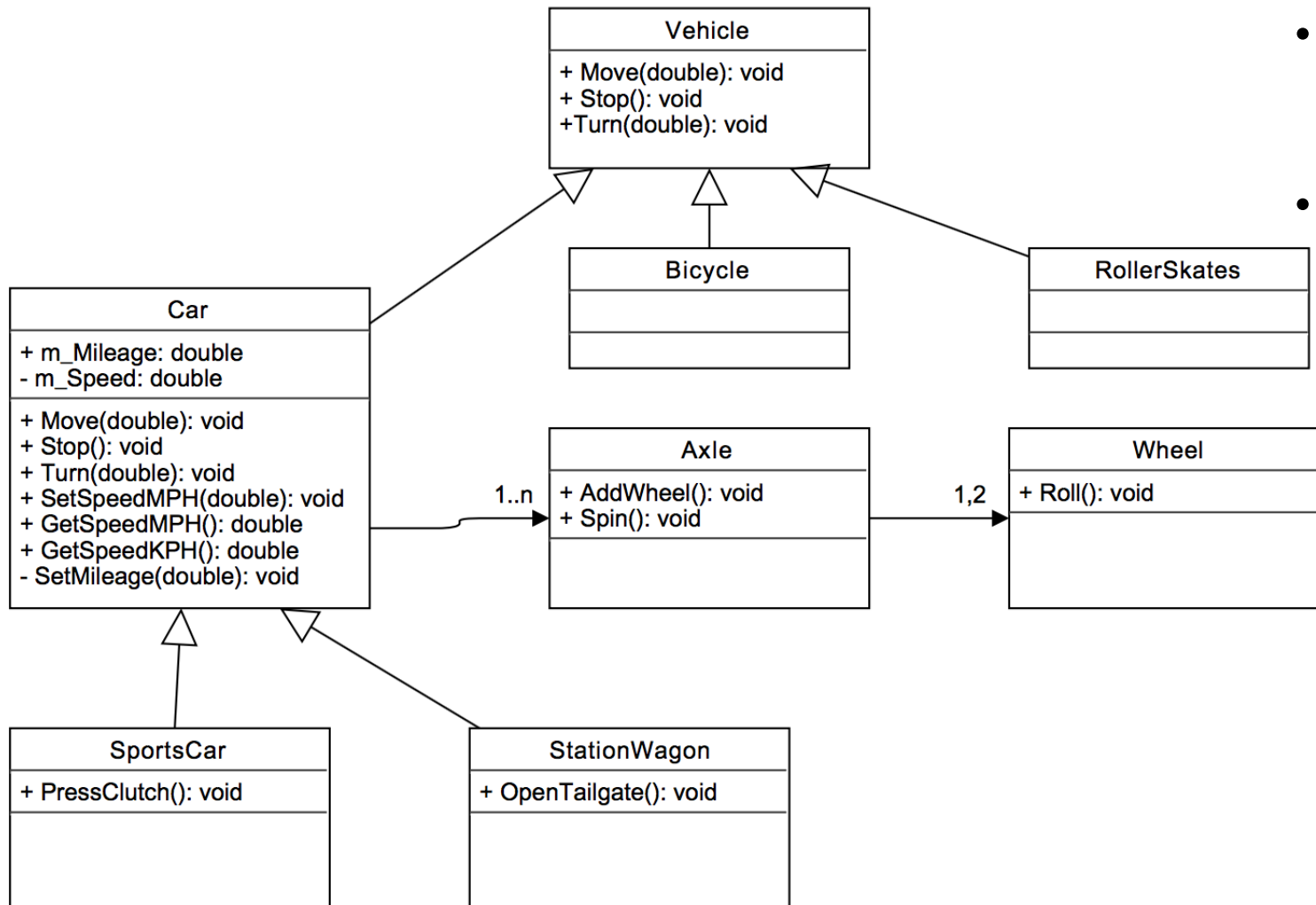


Generalization

- Specializing a superclass into one or more subclasses
 - A generalization set is a labeled group of generalizations with a common superclass
 - The label (sometimes called the discriminator) describes the criteria used in the specialization
 - Generalization in UML is implemented as inheritance in OO programming languages



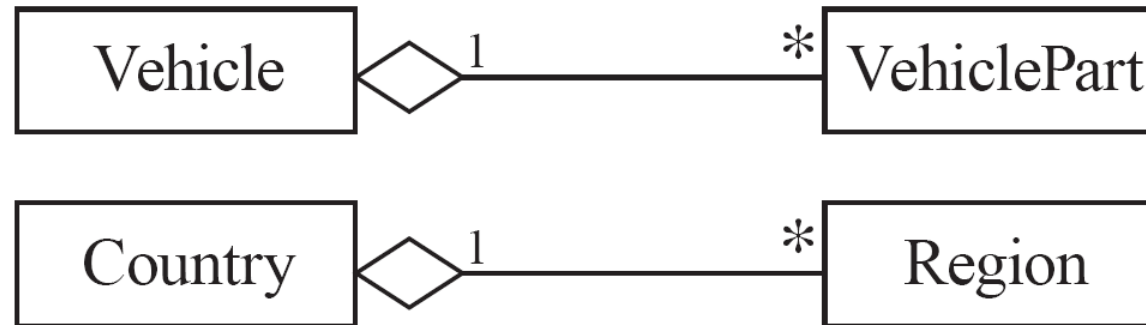
Class Activity



- The class **Vehicle** does not contain any attributes. (True/False)
- **SportsCar** has a method called `AddWheel()`. (True/False)

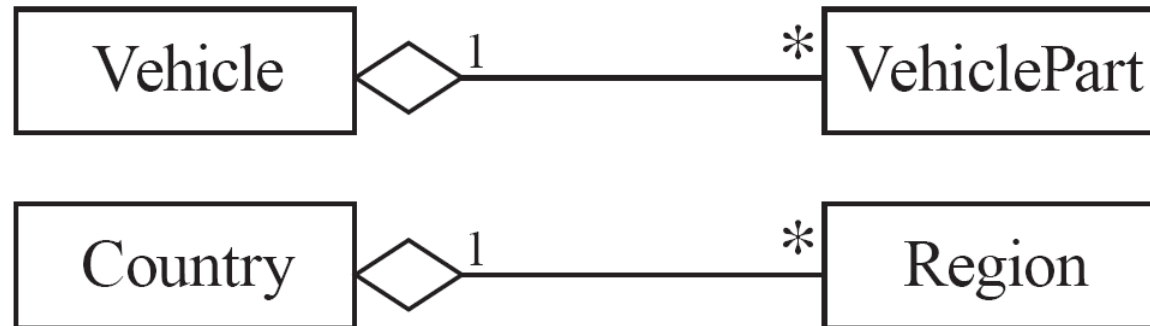
More Advanced Features: Aggregation

- Aggregations are special associations that represent ‘**part-whole**’ relationships.
 - The ‘whole’ side is often called the *assembly or the aggregate*
 - This symbol is a shorthand notation association named isPartOf



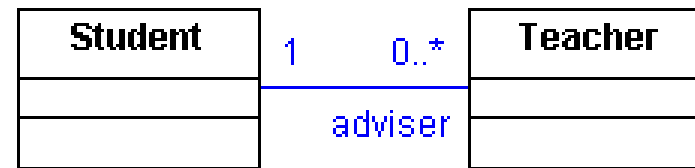
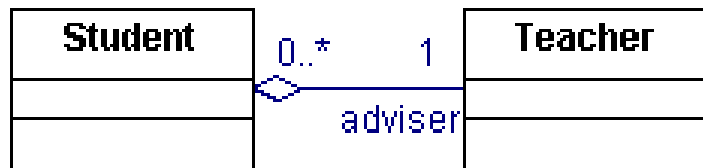
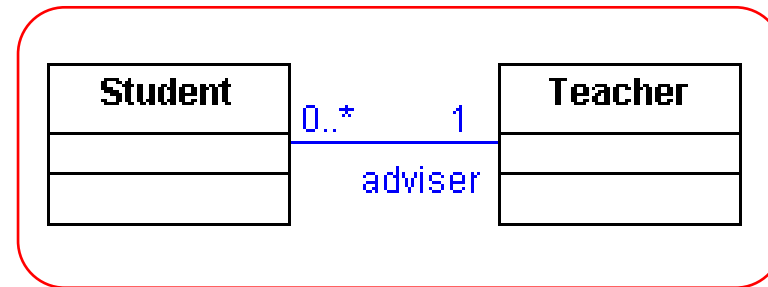
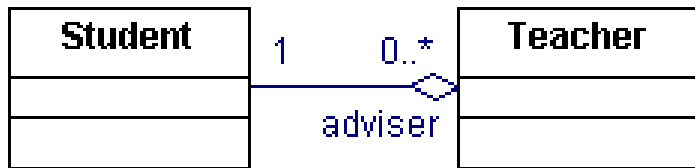
When to use an aggregation

- As a general rule, you can mark an association as an aggregation if the following are true:
 - You can state that
 - the parts 'are part of' the aggregate
 - or the aggregate 'is composed of' the parts
 - When something owns or controls the aggregate, then they also own or control the parts



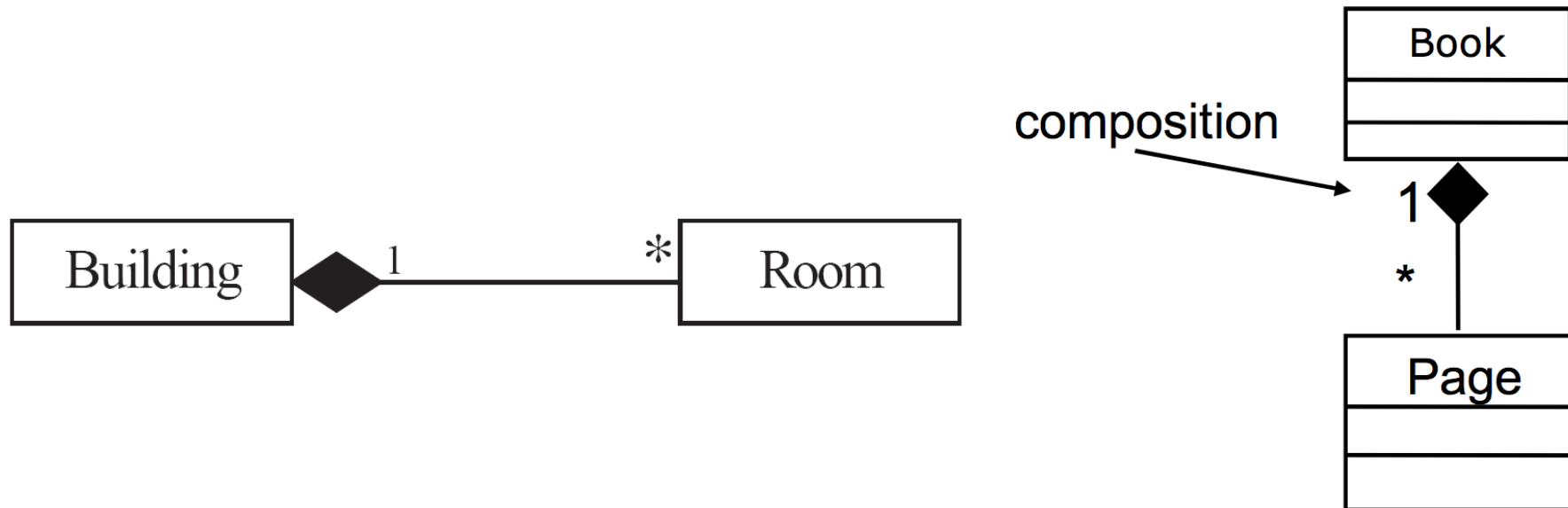
Class Diagrams

- Every student at TU will be advised by one teacher. Some teachers advise many students and some advise none. Which of the following class diagrams most clearly represents that student-teacher relationship?



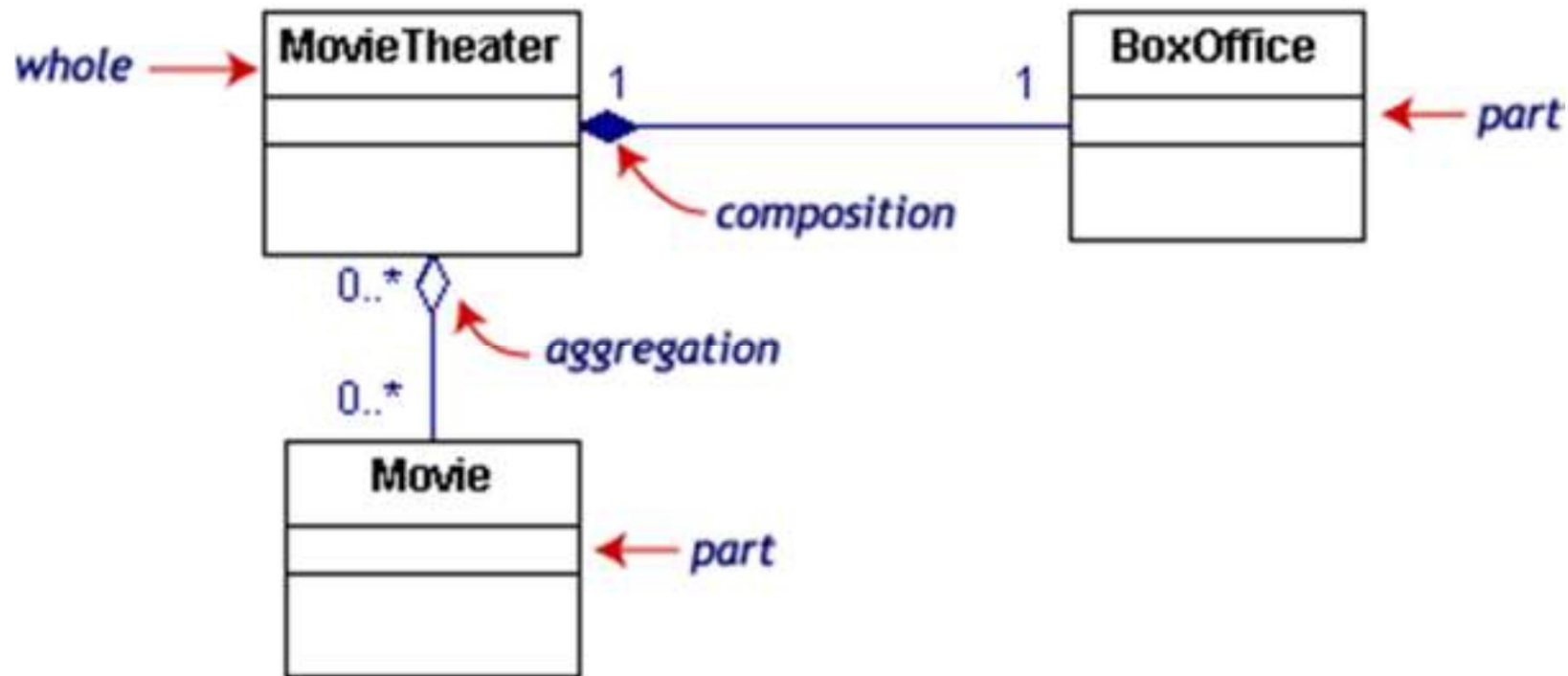
Composition

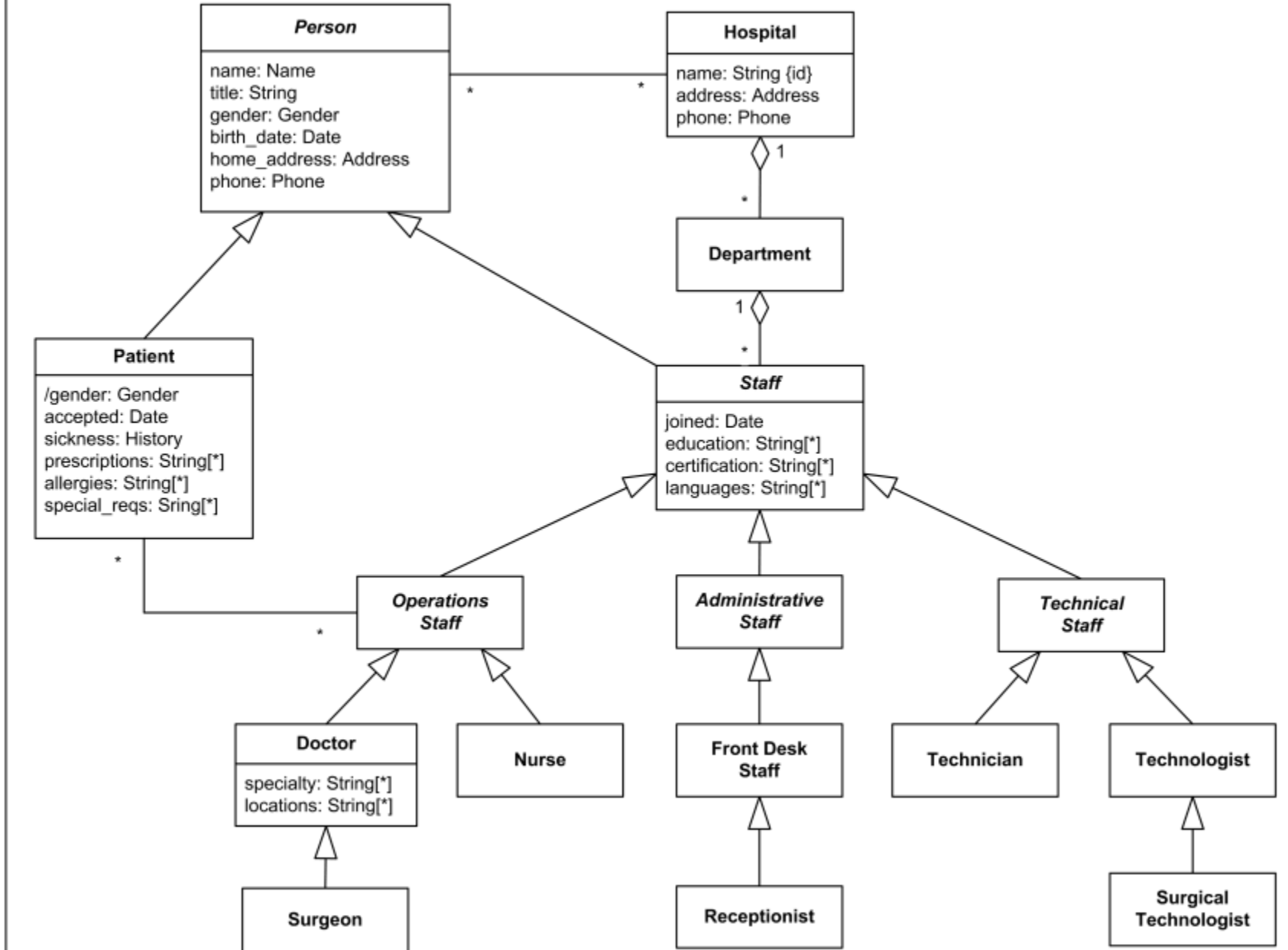
- A composition is a strong kind of aggregation
 - each part belongs to the whole that it is a part of, and is part of no other entity
 - each part is a necessary part of the whole
 - if the aggregate is destroyed, then the parts are destroyed as well
 - In other word, the part is assumed to belong only to that particular object, and is expected to “live and die” with it.



Composition vs Aggregation

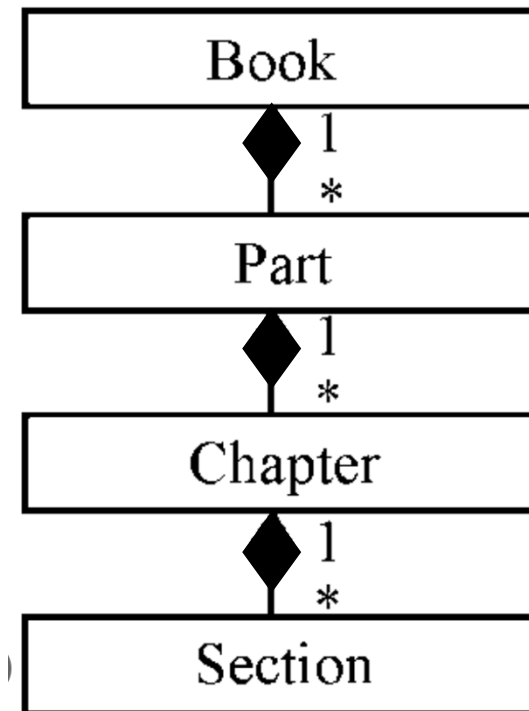
- If the movie theater goes away
 - so does the box office => composition
 - but movies may still exist => aggregation





Class Activity

- Draw a class diagram representing a book defined by the following statement: “A book is composed of a number of parts, which in turn are composed of a number of chapters. Chapters are composed of sections.” Focus only on classes and relationships.



Sequence Diagrams

Sequence Diagrams

- Used during analysis
 - To refine use case descriptions
 - To find additional objects (“participating objects”)
 - To discover responsibilities (methods in Java) a class may need to have
 - To define how a system will behave
- Used during system design
 - to refine subsystem interfaces

Why sequence diagrams?

Can we just code it?

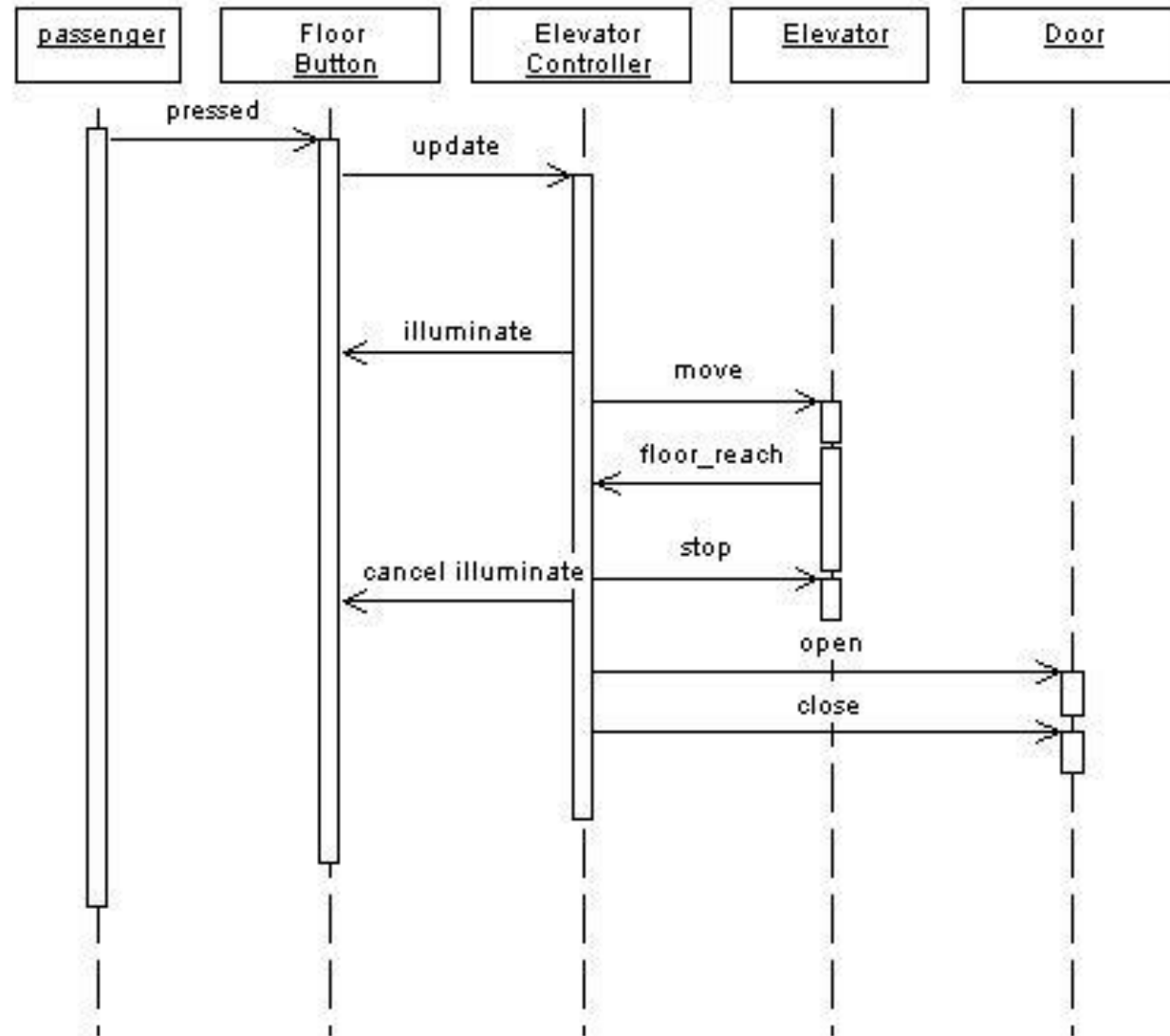
- A good sequence diagram is still a bit above the level of the real code (not all code is drawn on diagram)
- sequence diagrams are language independent, can be implemented in different languages;
- non-technical people can also understand/draw sequence diagrams;
- easy to do as a team;
- can see many objects at a time on same page;

Sequence Diagrams

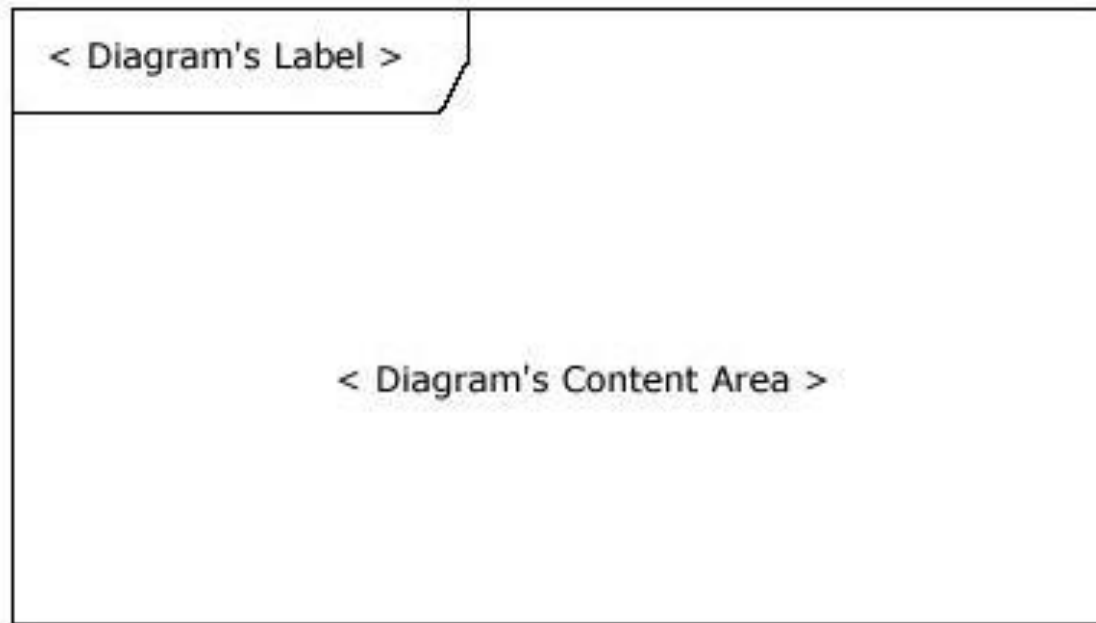
- A sequence diagram captures a “**snapshot**” of the program execution for accomplishing a single use case.
- Thus, a line coming from the **left edge of the frame** represents the **method to be called**, and the line **going out to the left edge** indicates the **returned results**.

Sequence Diagrams – Elevator example

Serving Door Button



UML Frame Element



- **Diagram Labels:**

- use case – use case diagram
- cd – class diagram
- sd – sequence diagram

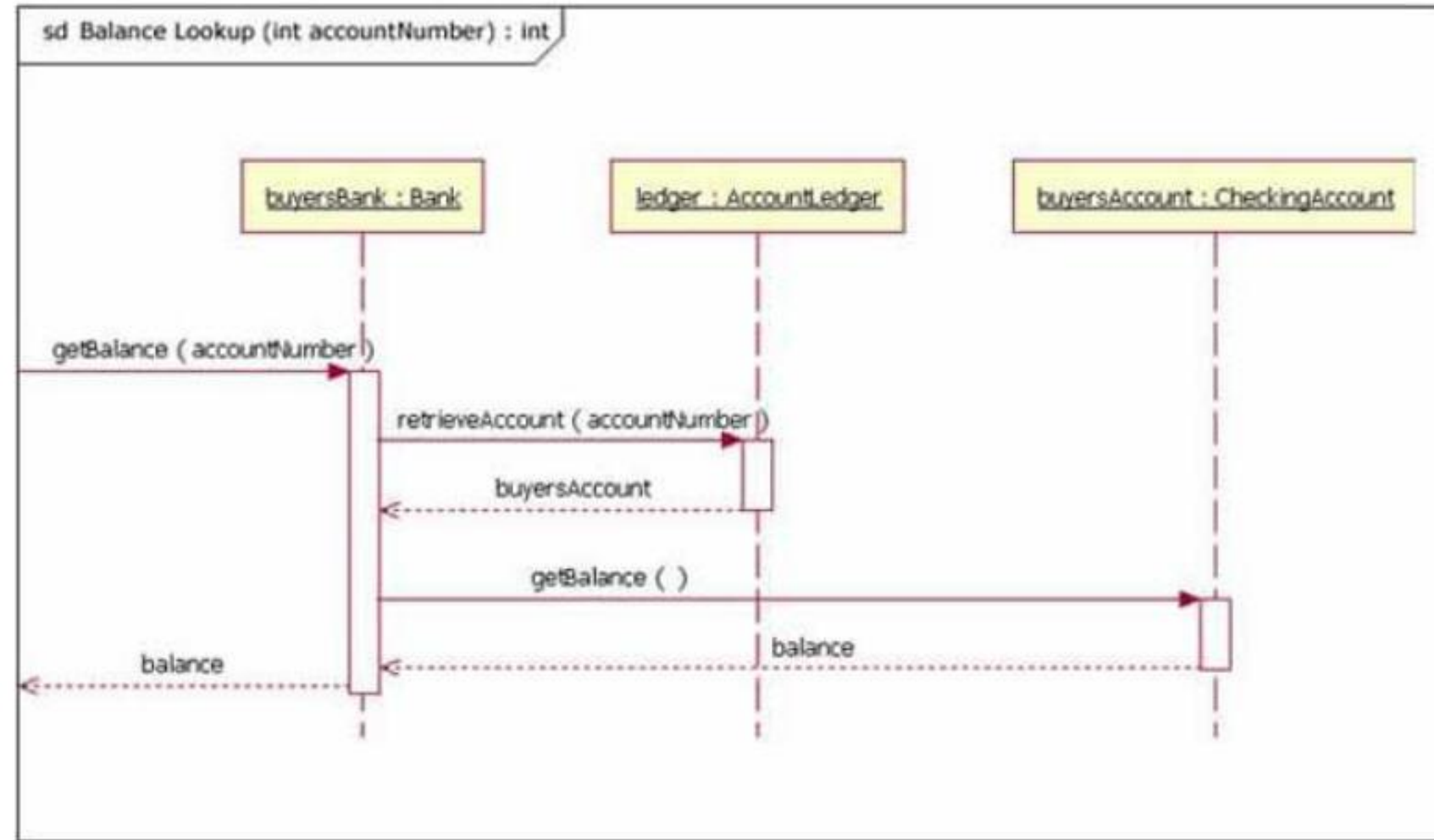
The Diagram Label indicates what type of diagram it is, for example, Use Case, Class Diagram or Sequence Diagram

UML Frame Element

- The following diagram is labeled “sd” for sequence diagram.

It is also named and described as accomplishing use case “Balance Lookup”.

Figure 2: A sequence diagram that has incoming and outgoing messages

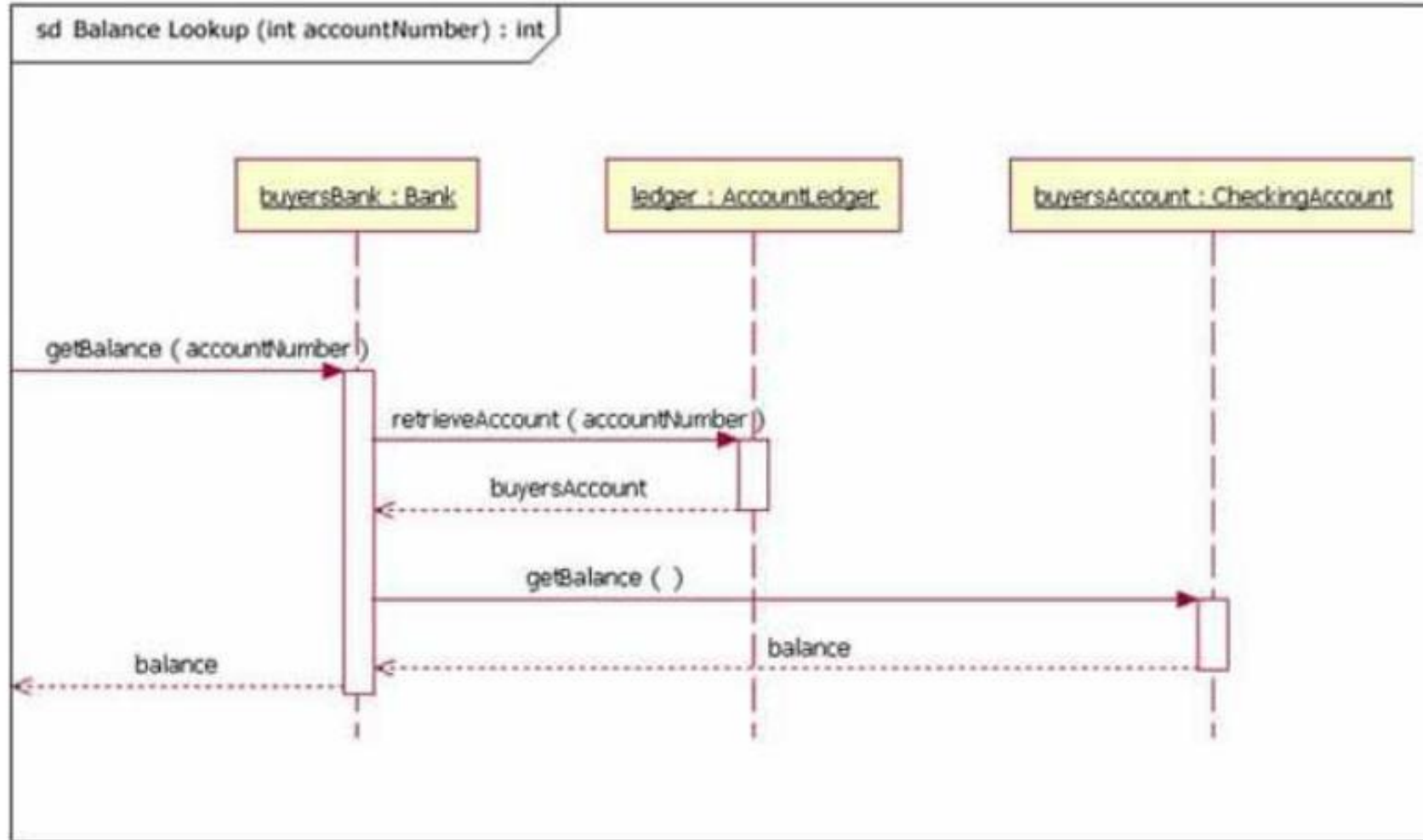


Sequence Diagrams

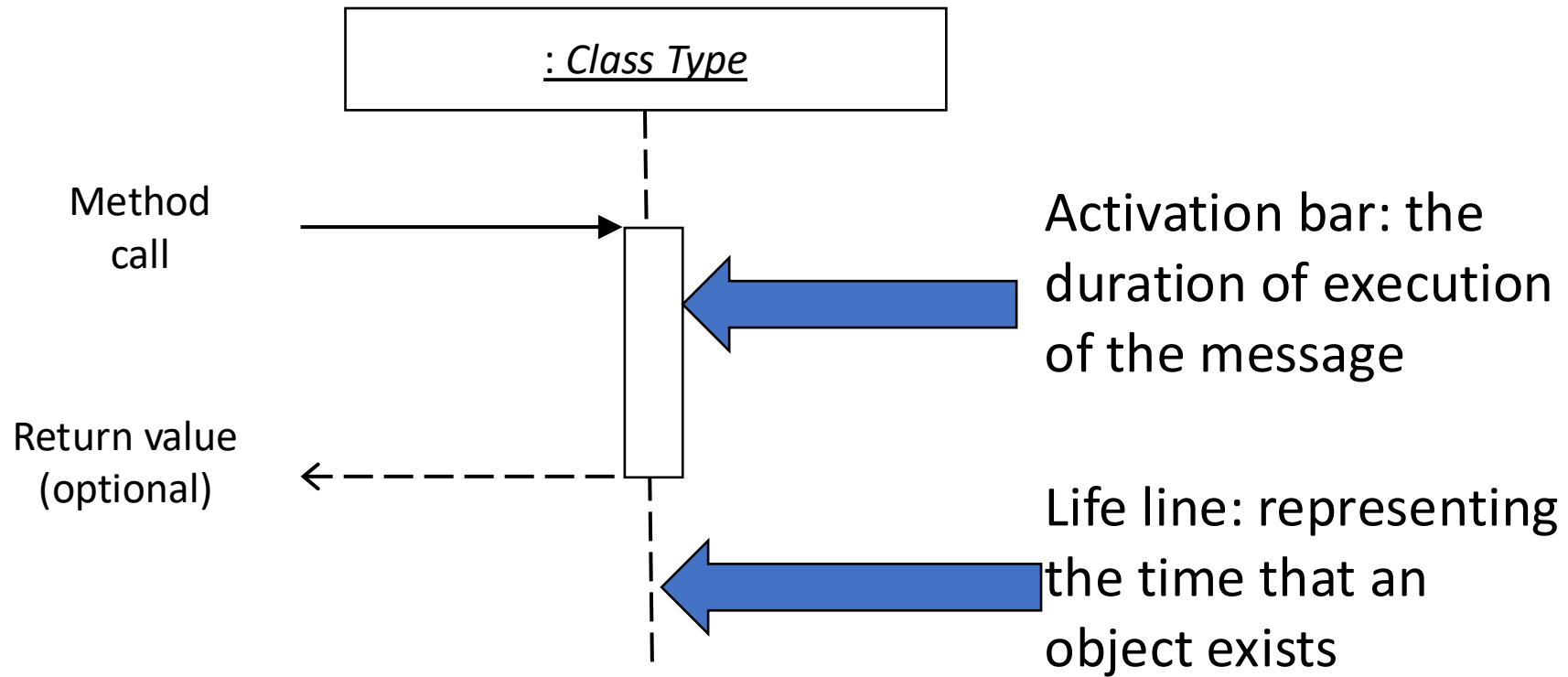
- A sequence diagram **captures a “snapshot”** of the program execution for accomplishing a single use case.
- Thus, a **line coming from the left edge** of the frame represents the **method to be called**, and the **line going out to the left edge** indicates the **returned results**.

Sequence Diagrams

Figure 2: A sequence diagram that has incoming and outgoing messages



Basic Syntax of UML Sequence Diagrams

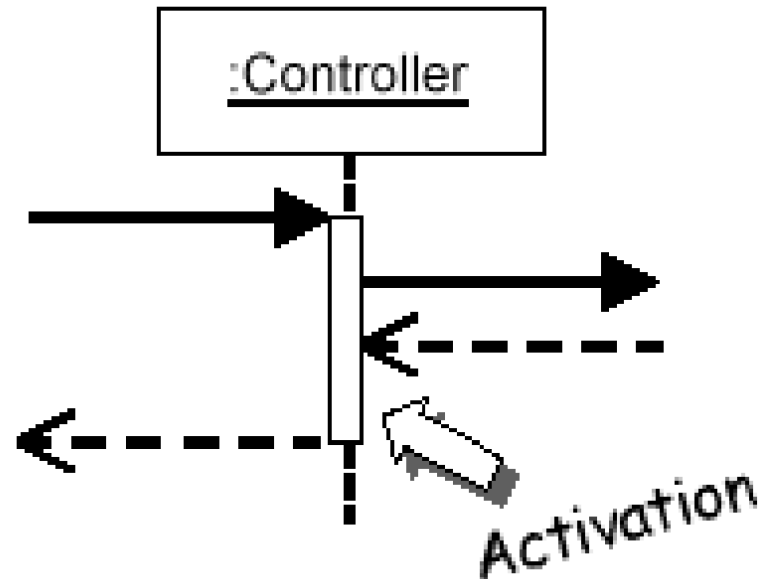


Note that when the class type is **underlined**, it means that what is involved in the interactions is an object instance of the class. (That is, that instance methods are being called.)

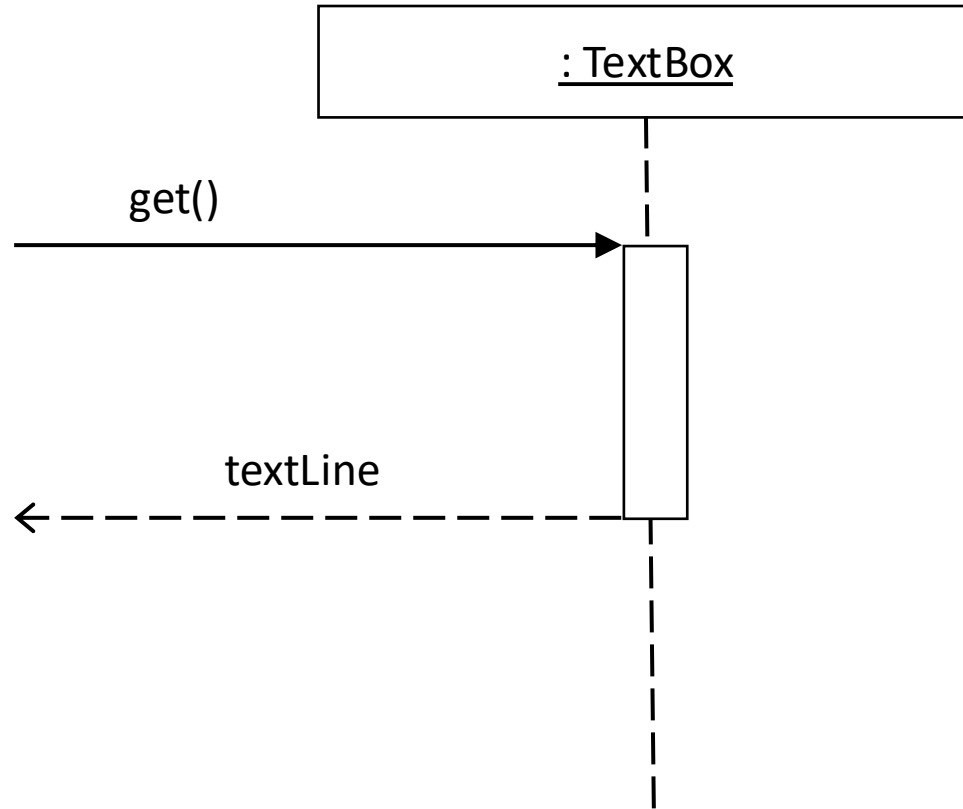
When a class name is **not underlined**, it means that the **static methods** of the class are accessed.

Activation

- Thick box over object's life line;
 - drawn when object's method is on the stack
- either that object is **running** its code, or
- it is on the stack **waiting** for another object's method to finish

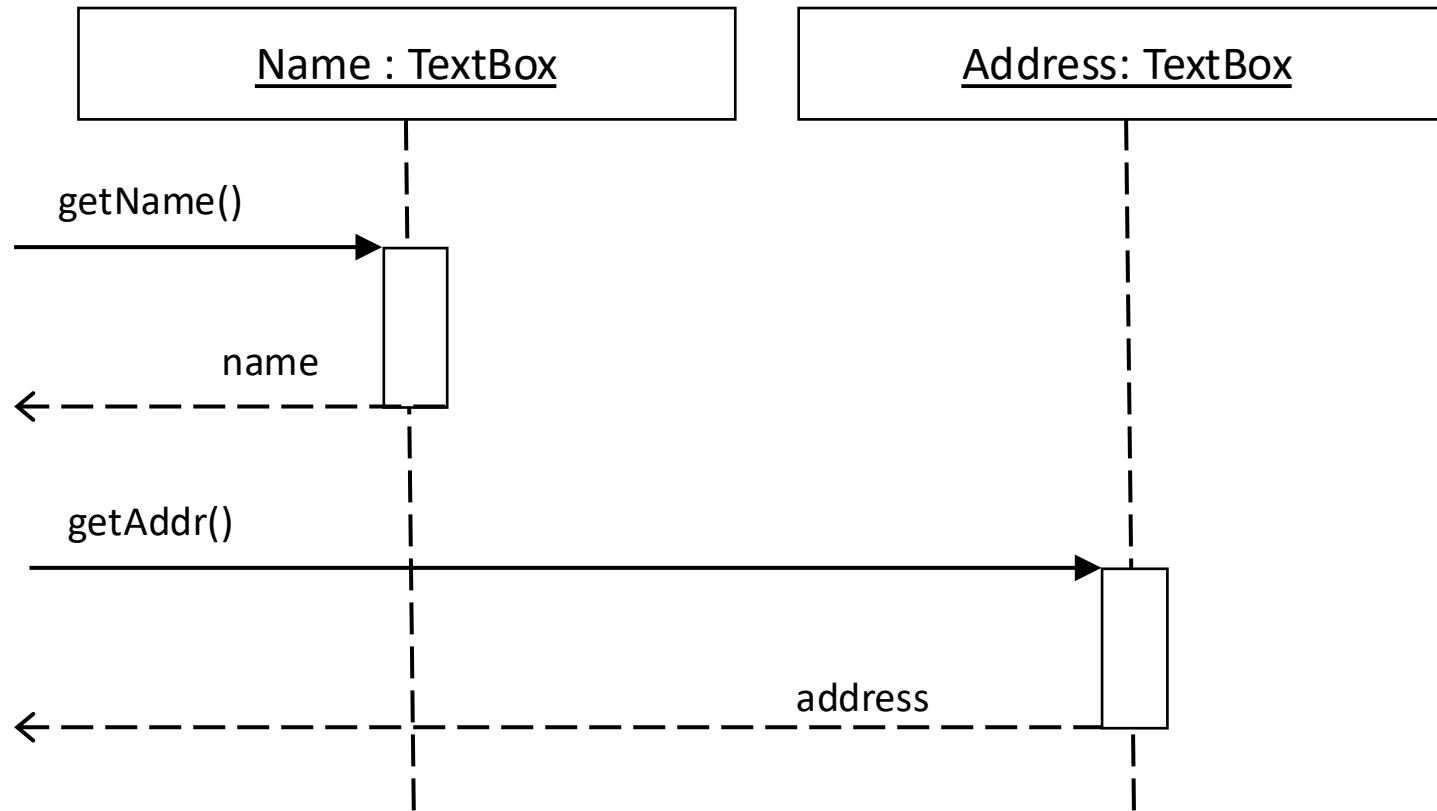


Objects



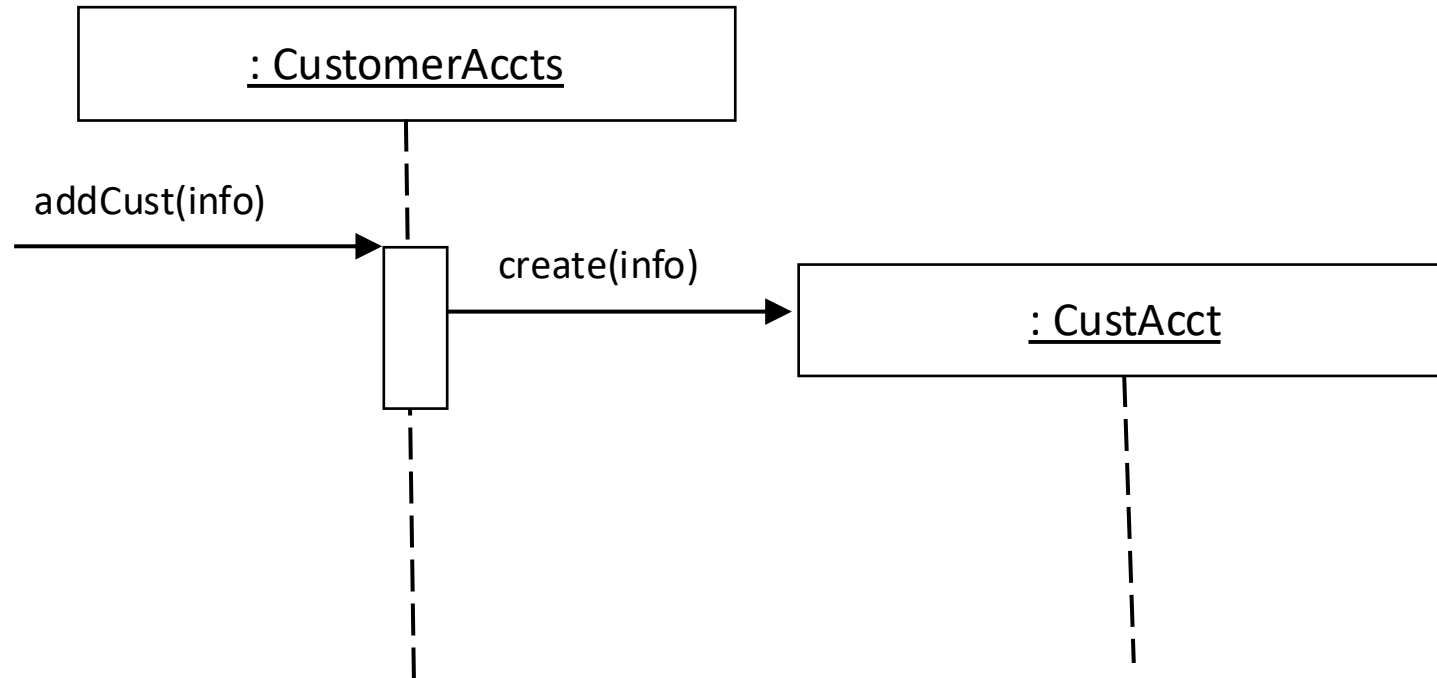
If there is only one object of type `TextBox`, then this notation is OK. But if there is more than one object of the same type, then we need to **distinguish** them.

Multiple Objects of same class



Object creation

- What if an object is created as a result of the object interaction?

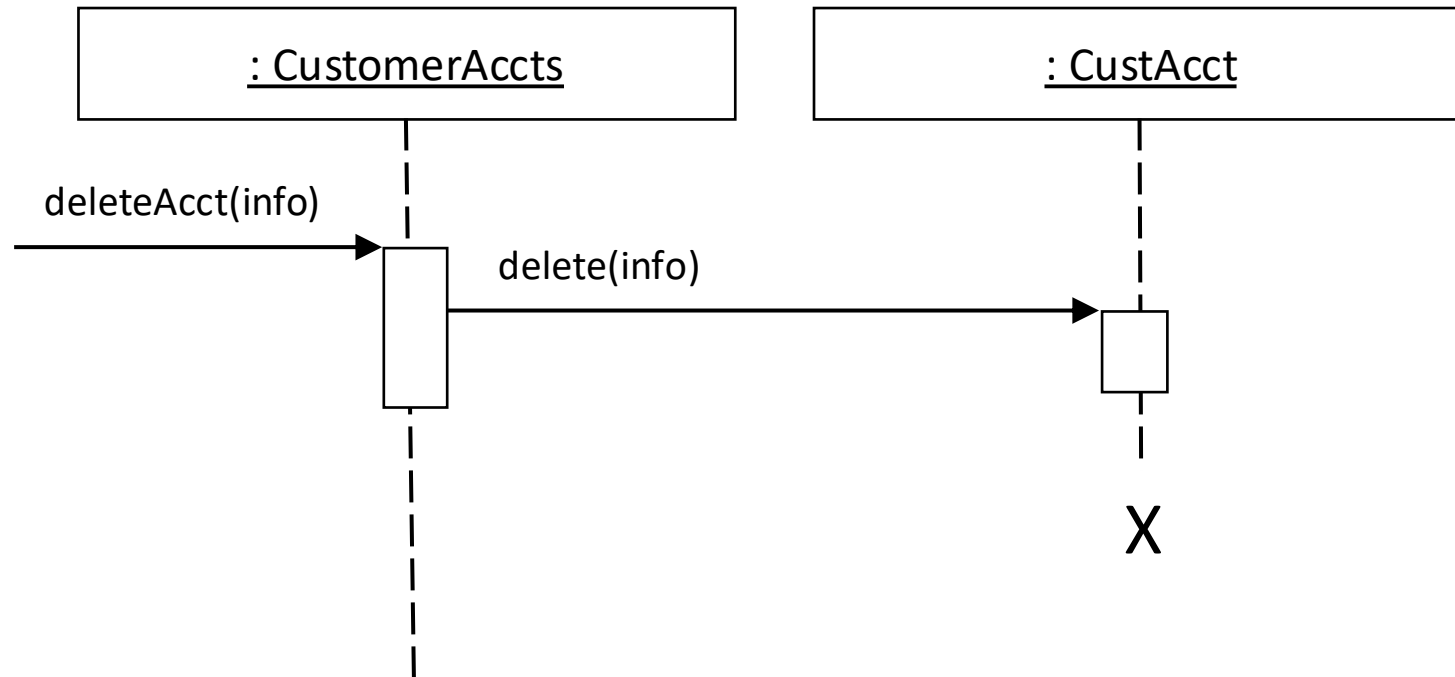


Method name of create indicates that a **constructor** or some other object creation method is called.

Object deallocation

- Sometimes an **object is deallocated** as a result of object interactions.

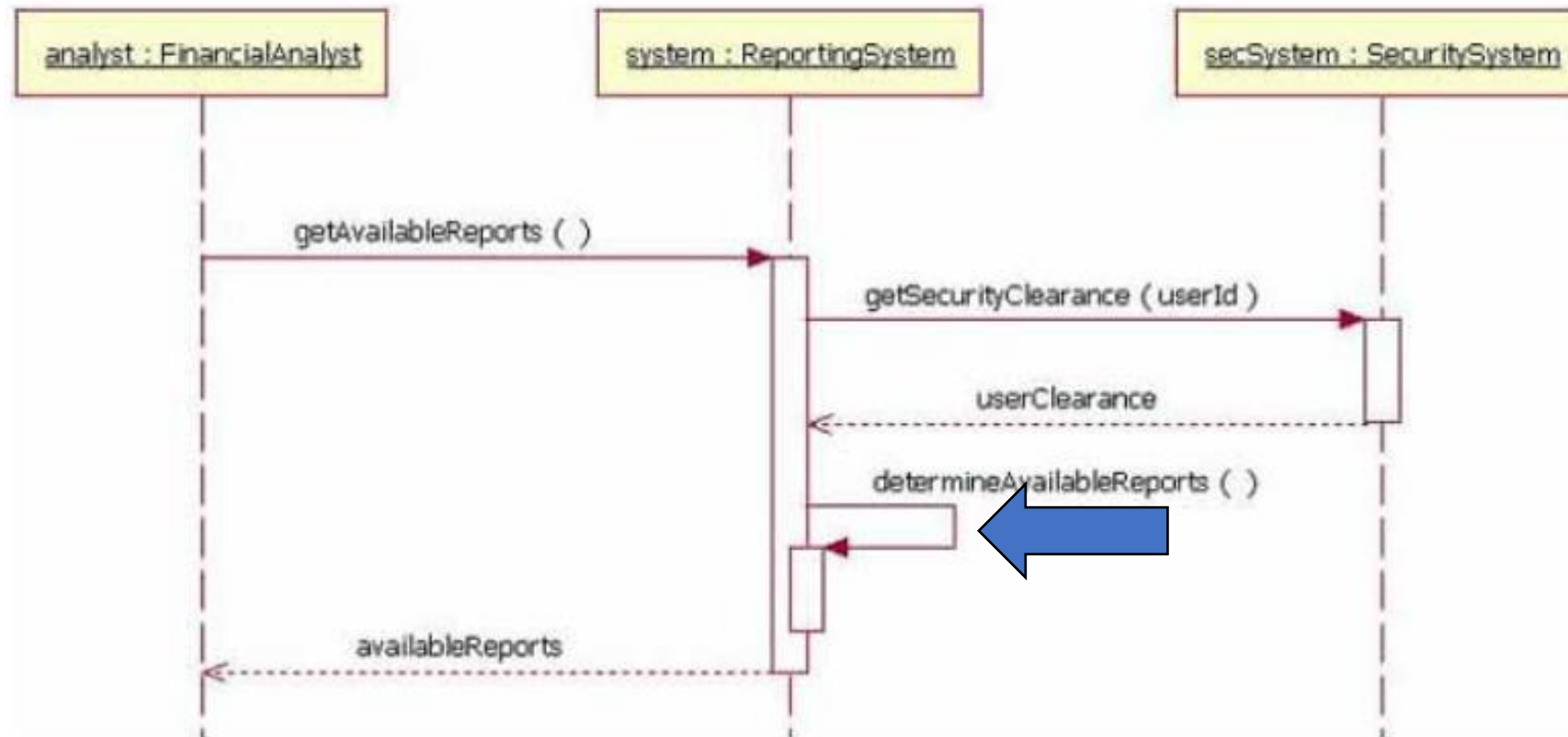
This is represented by **placing an X** at the **end of the life line**.



Method calls – to similar objects

- What if a method calling another method of the same object?

Figure 5: The system object calling its determineAvailableReports method



Conditions

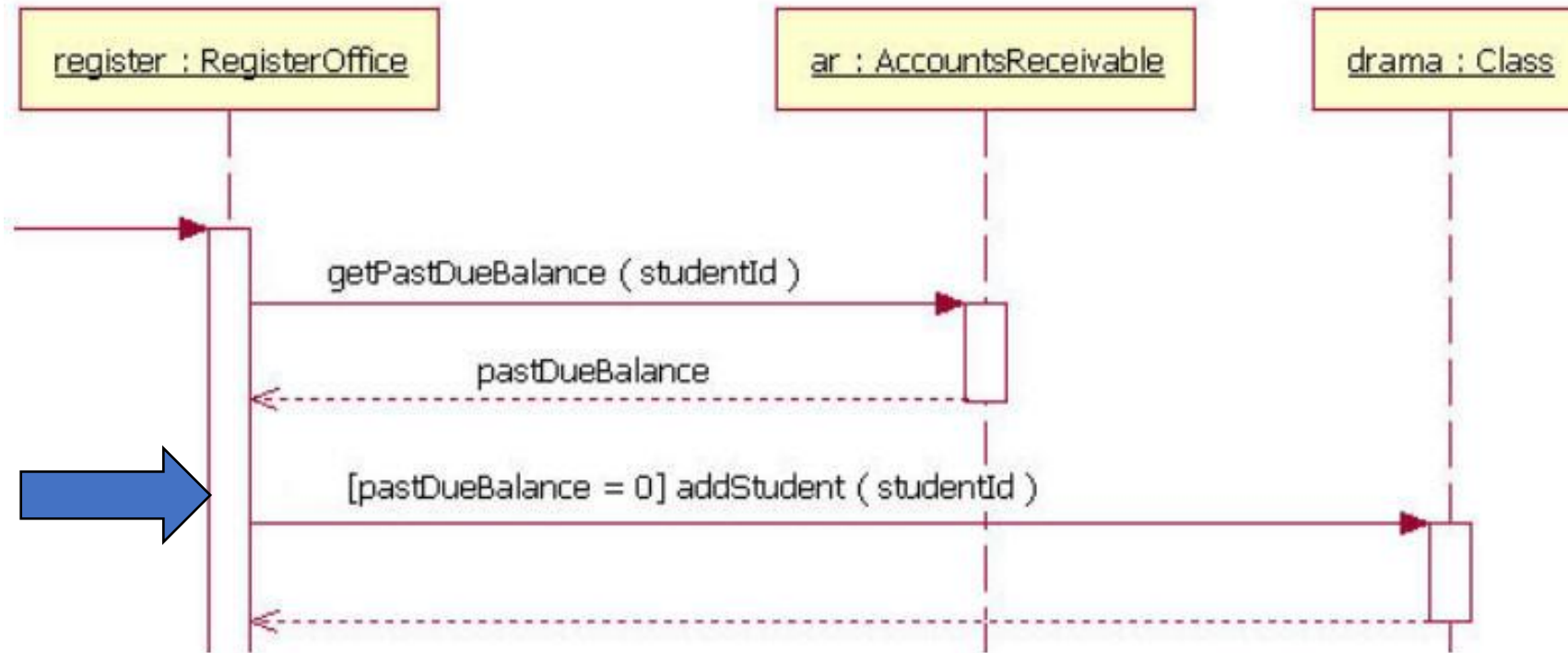
- **What if there are conditions?**

- These can usually go three ways:
 - Conditional statement – with single instruction to execute (*if with single instruction*)
 - Conditional statement – with multiple instructions to execute (*if with multiple instructions*)
 - Conditional statement – with 2 alternatives set of multiple instructions to execute (*if-else*)

Conditions – single instruction

- What if there are conditions?

Figure 7: A segment of a UML 1.x sequence diagram in which the addStudent message has a guard



A **guard** is a way to indicate that a specific method call is made **conditionally**.

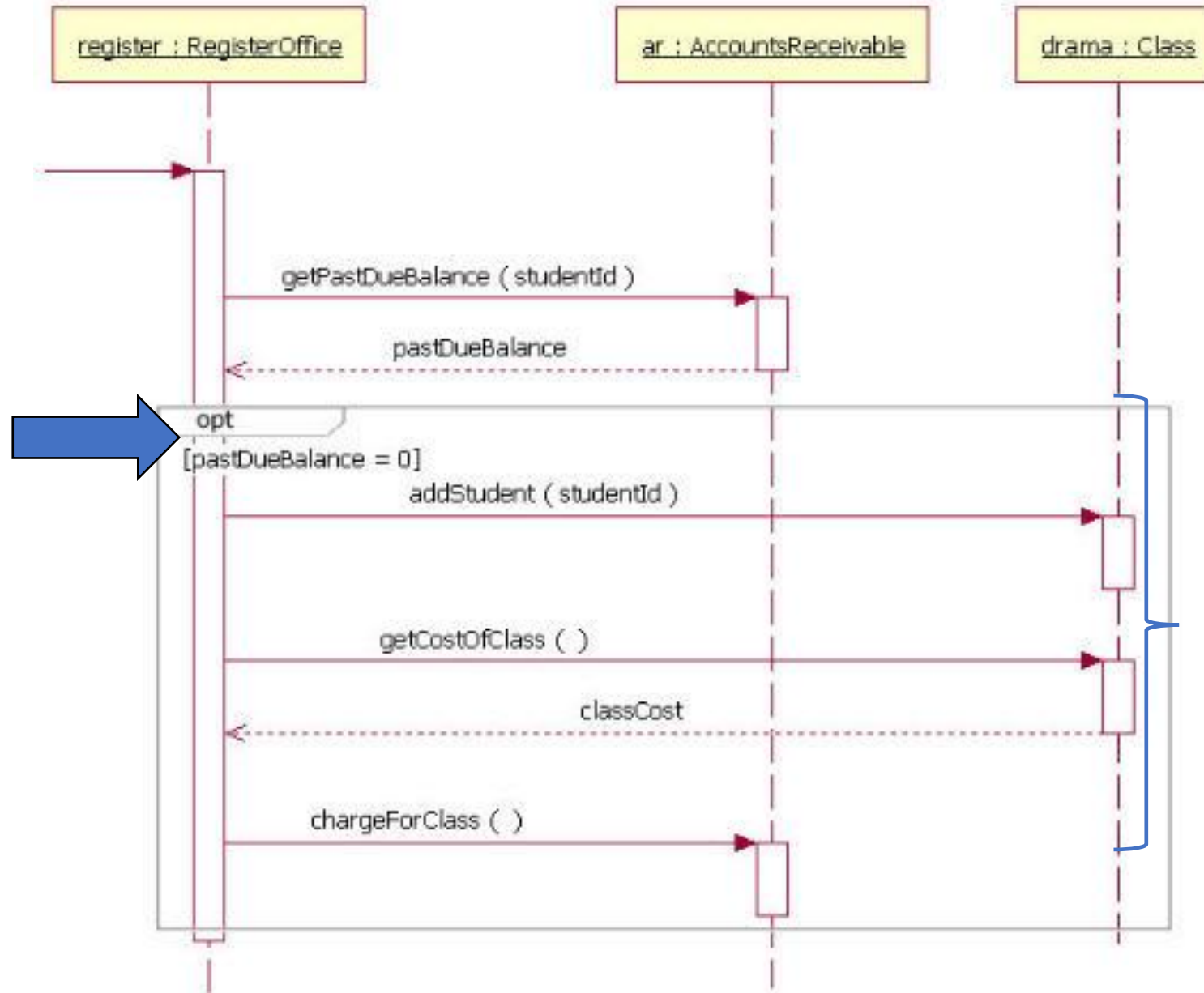
In the above, a student is added to a class only if their past due balance is 0.

Conditions – multiple instructions

- There are times, however, when a set of instructions need to be conditionally executed.

The if form of execution in UML is called opt.

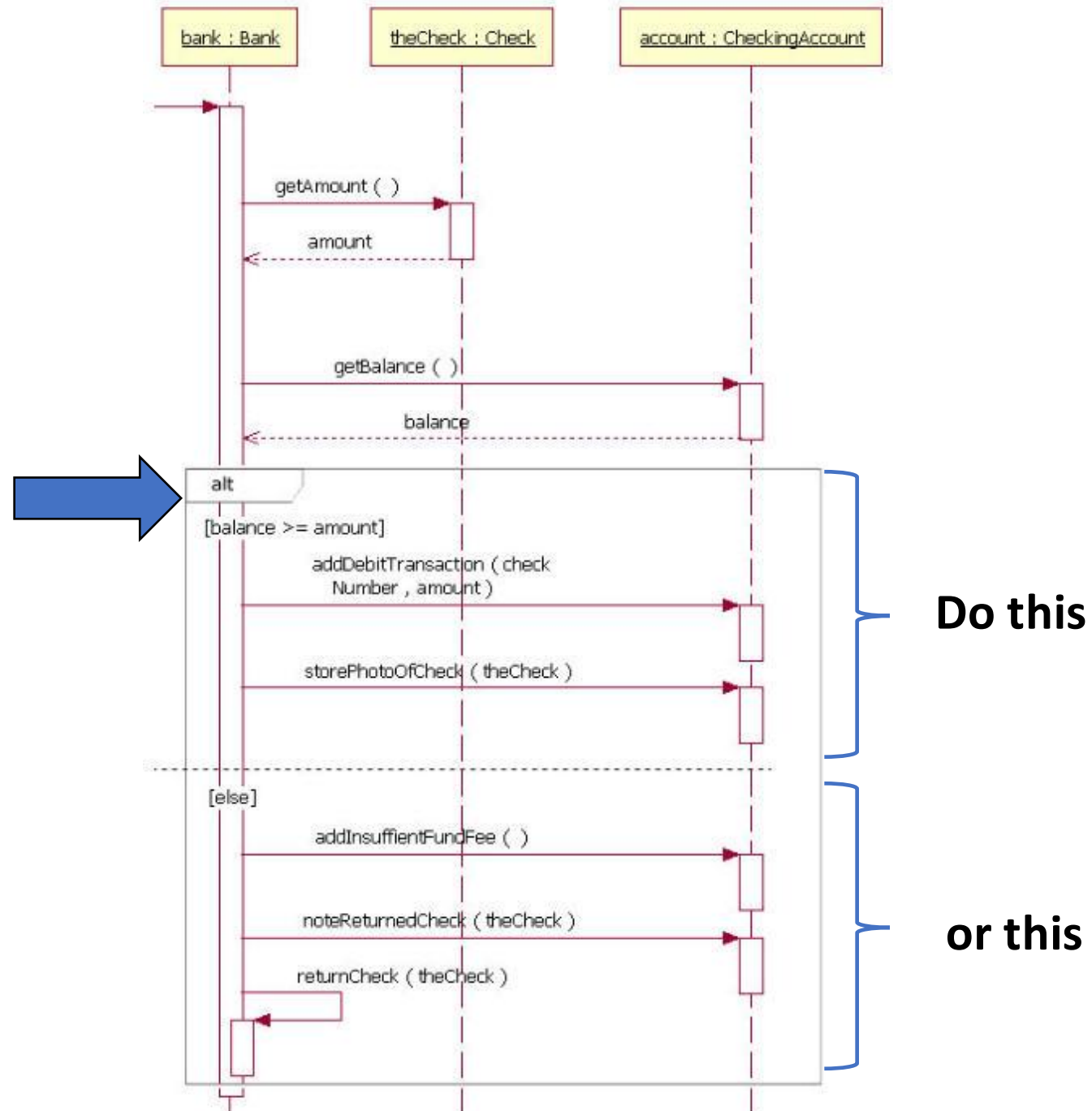
Figure 9: A sequence diagram fragment that includes an option combination fragment



Conditions – multiple instructions (if-else)

The if-else form of execution in UML is called alt.

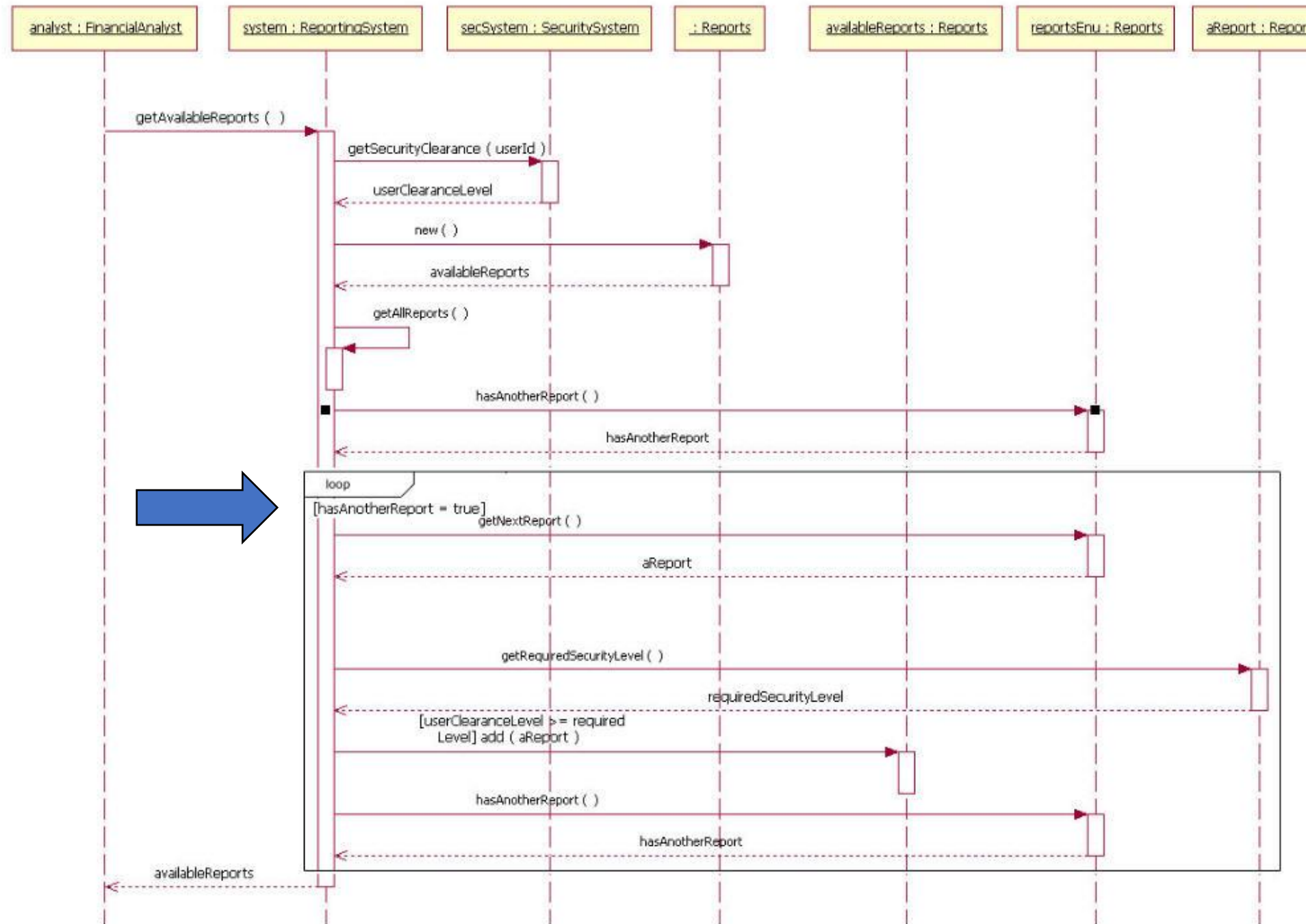
Figure 8: A sequence diagram fragment that contains an alternative combination fragment



Loops

The iterative form of execution in UML is called loop.

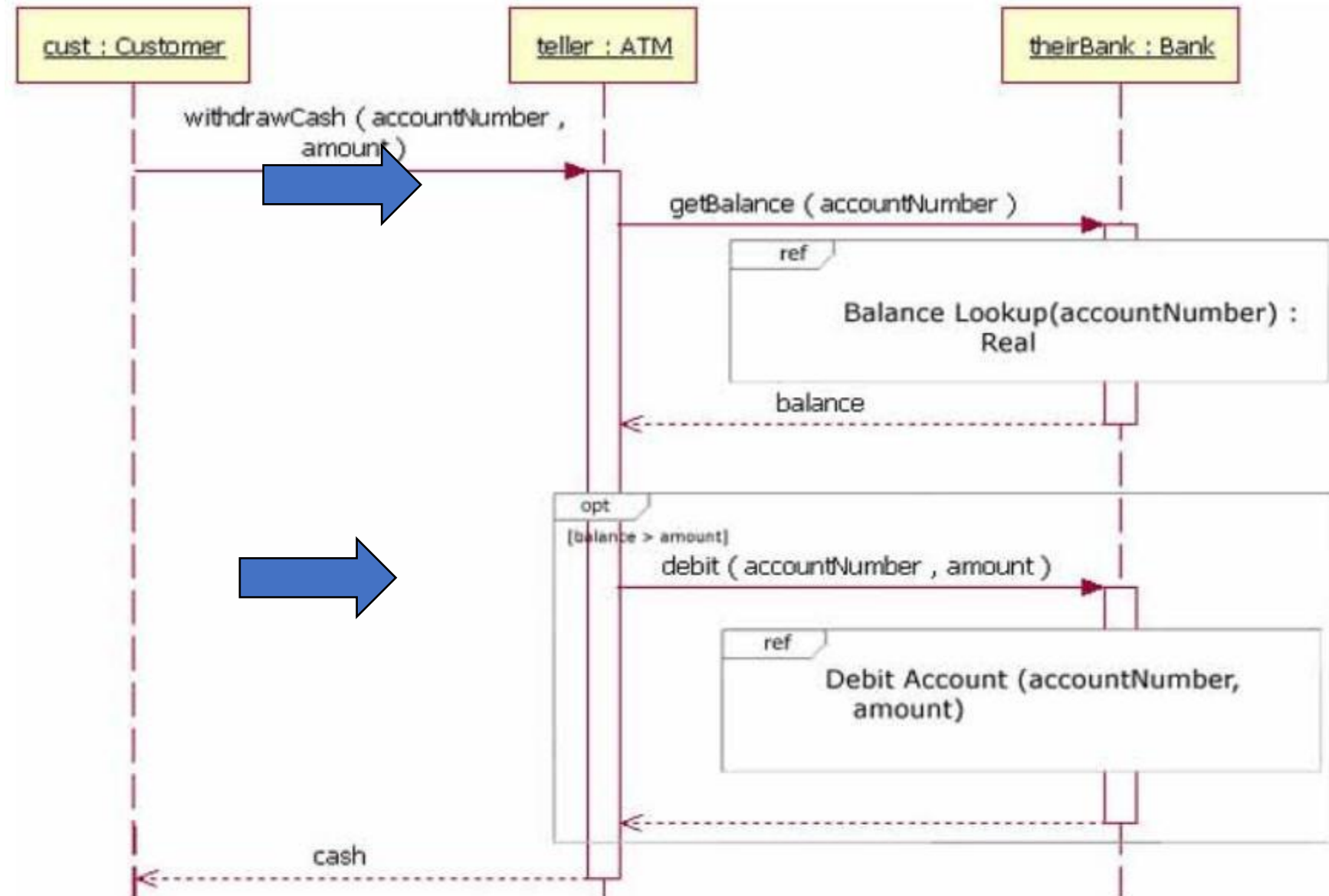
Figure 10: An example sequence diagram with a loop combination fragment



References

- If a given sequence diagram contains too much detail to put on one page, can use the ref ...

Figure 11: A sequence diagram that references two different sequence diagrams



Why sequence diagrams?

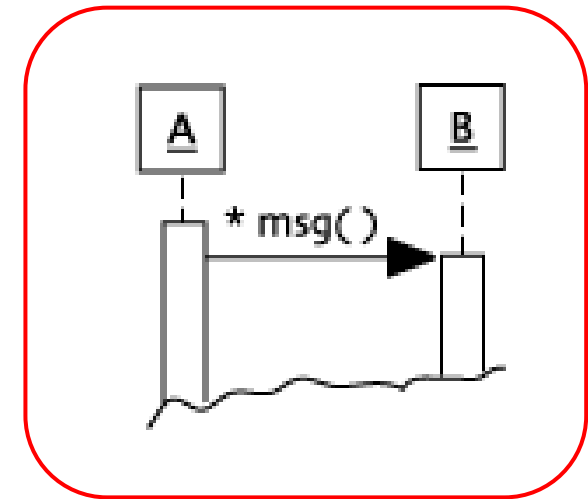
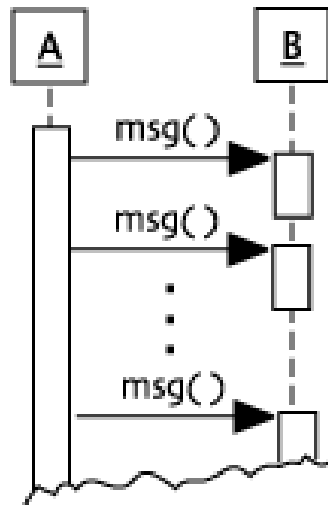
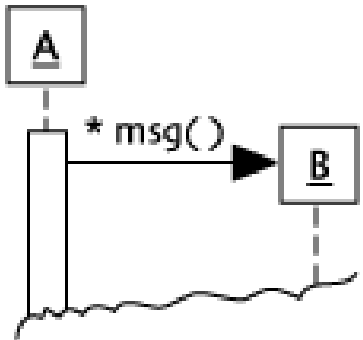
Can we just code it?

- A good sequence diagram is still a bit above the level of the real code (not all code is drawn on diagram)
- sequence diagrams are language independent, can be implemented in different languages;
- non-technical people can also understand/draw sequence diagrams;
- easy to do as a team;
- can see many objects at a time on same page;

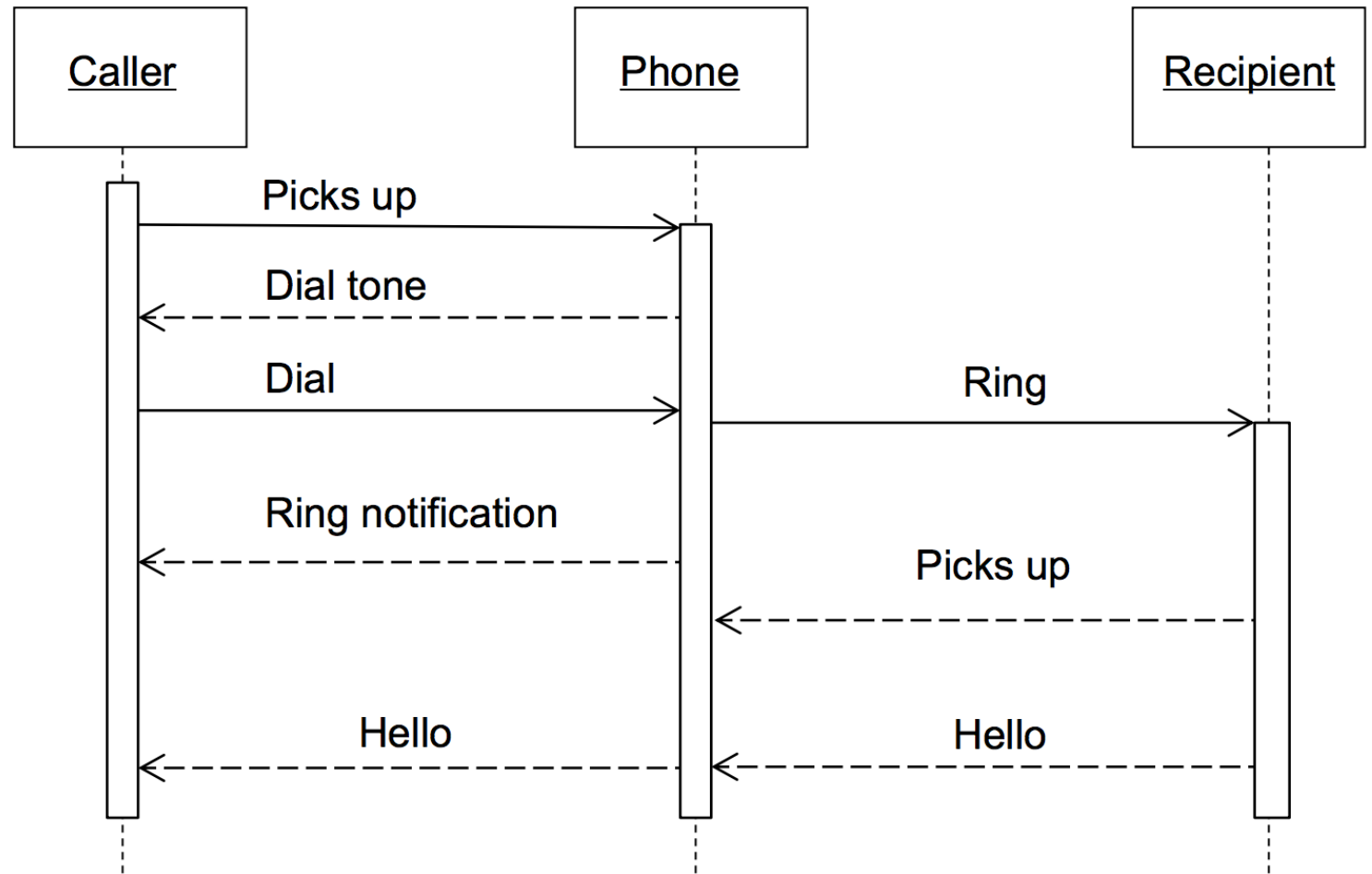
More Example Interaction Diagrams

Sequence Diagrams

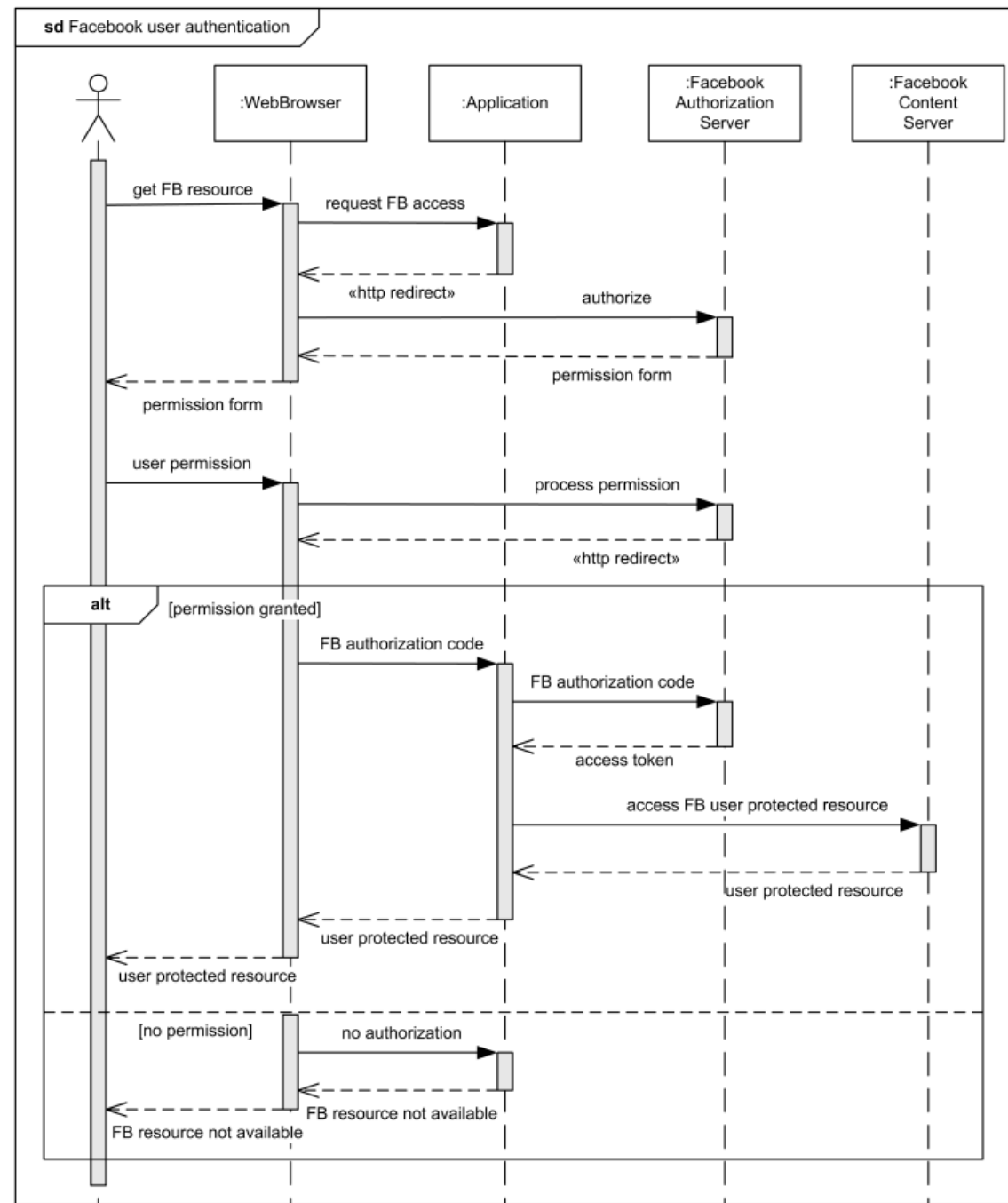
- Which of the following pieces of sequence diagrams represents this action:
"Object A sends object B several messages named msg."



Making a phone call



Facebook User Authentication



Sequence diagram example - Facebook User Authentication in a Web Application.

Class Activity

- Consider the sequence of steps needed to withdraw cash from a ATM.
- Following are the actors/components involved – person, ATM and bank system

Behavioral models

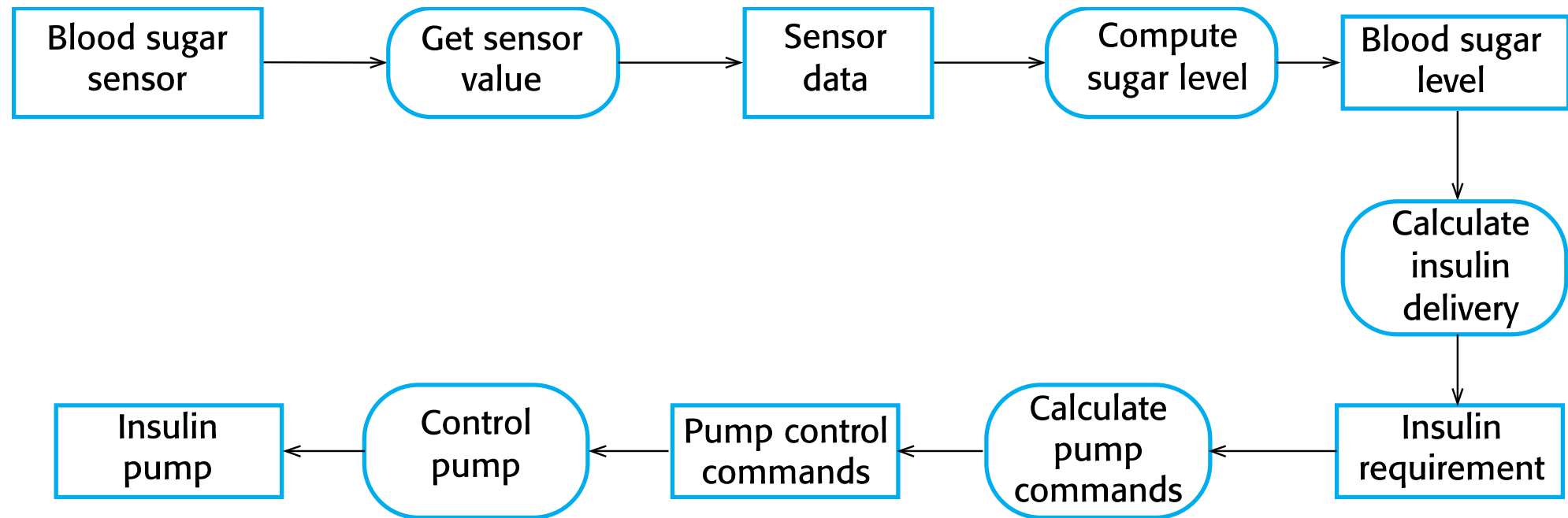
Behavioral models

- Behavioral models are models of the dynamic behavior of a system as it is executing.
 - They show what happens or what is supposed to happen
 - when a system responds to a stimulus from its environment.
- You can think of these stimuli as being of two types:
 - Data
 - Some data arrives that has to be processed by the system.
 - Events
 - Some event happens that triggers system processing. Events may have associated data, although this is not always the case.

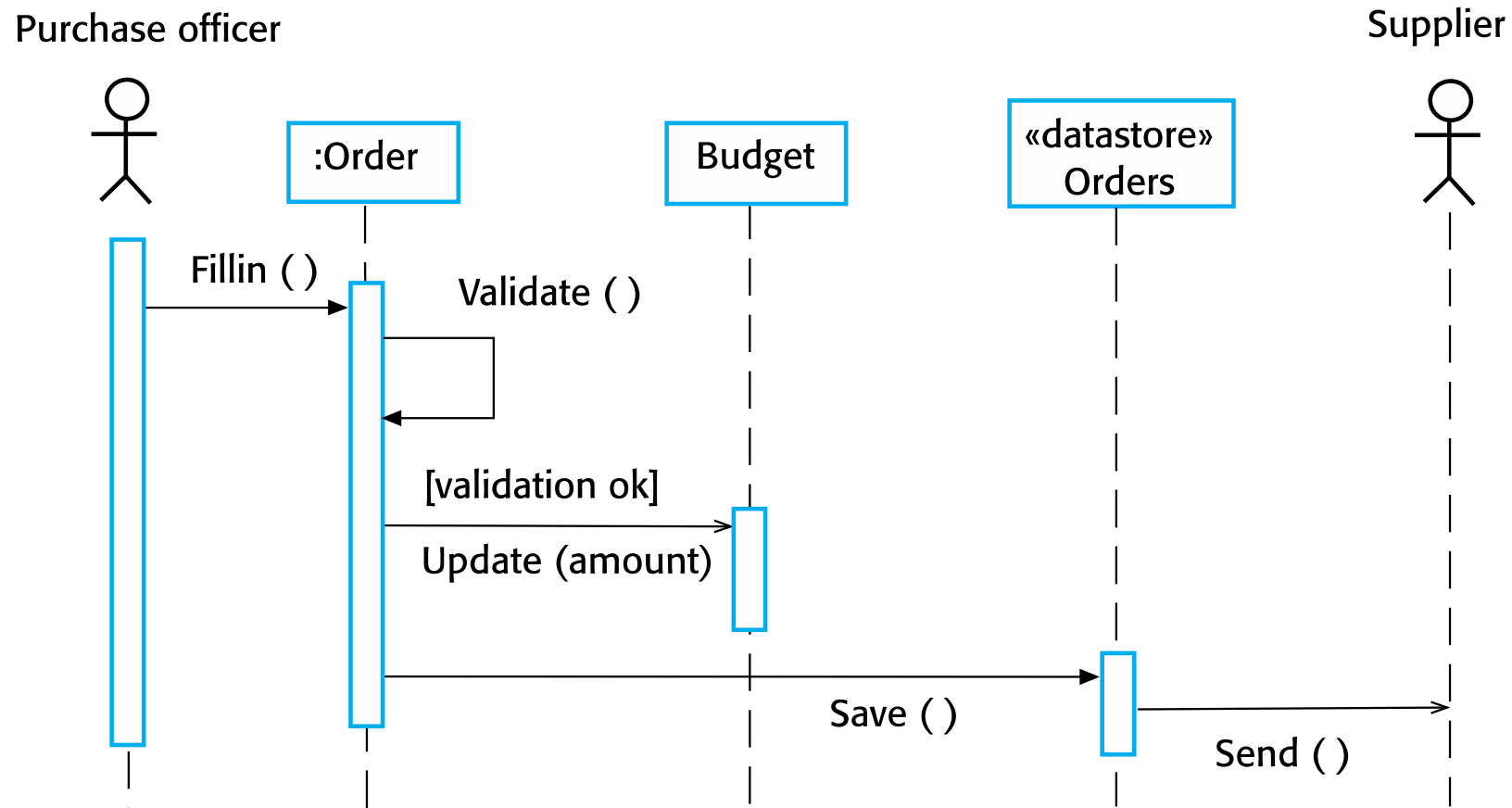
Data-driven modeling

- Many business systems are data-processing systems that are primarily driven by data
 - They are controlled by the data input to the system,
 - With relatively little external event processing.
- Data-driven models
 - show the sequence of actions involved in processing input data and generating an associated output.
 - Useful during the analysis of requirements
 - as they can be used to show end-to-end processing in a system.

Data-driven modeling: An activity model of the insulin pump's operation



Objects-driven modeling: Order processing



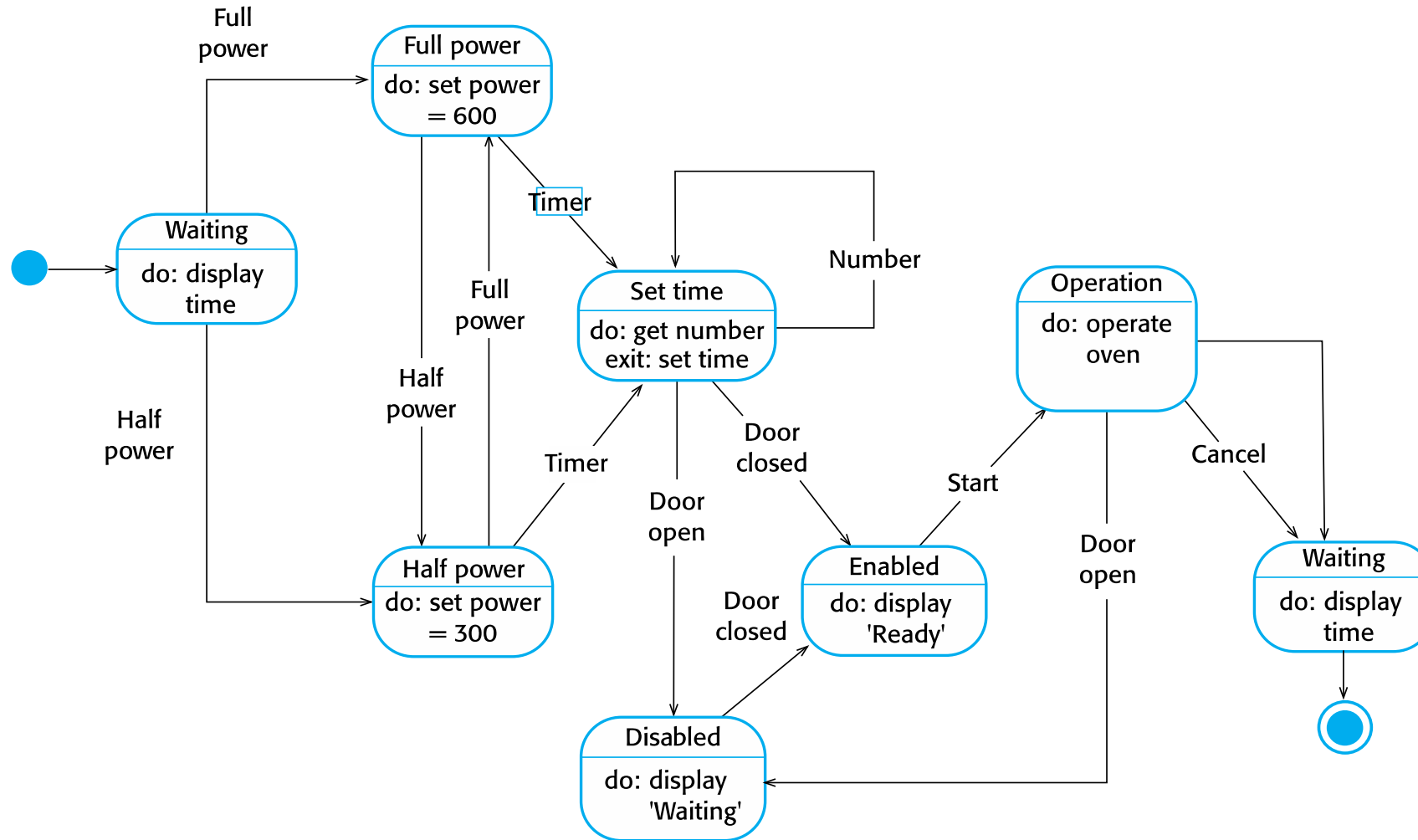
Event-driven Modeling

- Real-time systems are often event-driven, with minimal data processing.
 - For example,
 - a landline phone switching system responds to events such as
 - 'receiver off hook' by generating a dial tone.
- Event-driven modeling
 - shows how a system responds to external and internal events.
 - It is based on the assumption that a system has a finite number of states
 - and that events (stimuli) may cause a transition from one state to another.

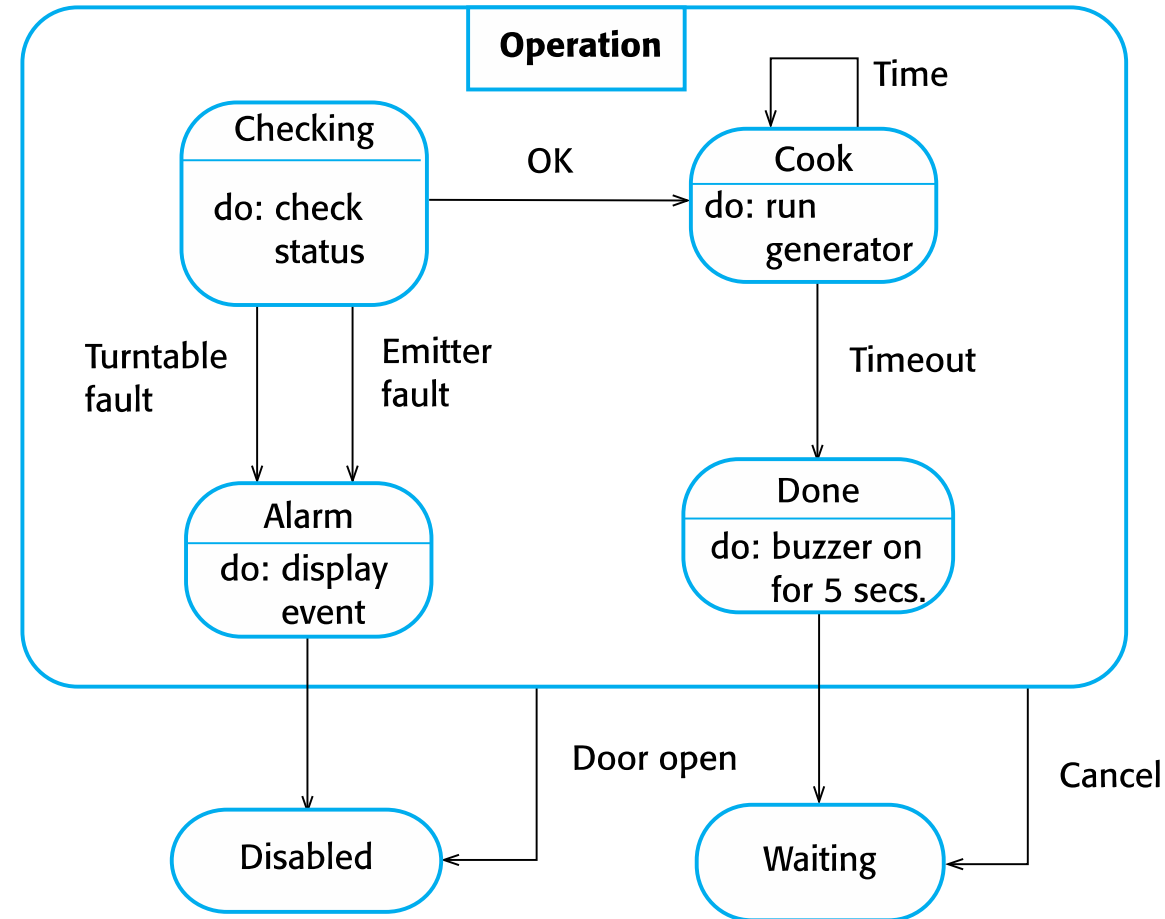
State Machine Models

- These model the behaviour of the system
 - in response to external and internal events.
 - often used for modelling real-time systems.
- State machine models show
 - system states as nodes and
 - events as arcs between these nodes.
 - When an event occurs,
 - the system moves from one state to another.
- Statecharts are an integral part of the UML and are used to represent state machine models.

State Diagram of a Microwave Oven



Microwave Oven Operation



States and stimuli for the microwave oven (a)

<u>State</u>	<u>Description</u>
Waiting	The oven is waiting for input. The display shows the current time.
Half power	The oven power is set to 300 watts. The display shows 'Half power'.
Full power	The oven power is set to 600 watts. The display shows 'Full power'.
Set time	The cooking time is set to the user's input value. The display shows the cooking time selected and is updated as the time is set.
Disabled	Oven operation is disabled for safety. Interior oven light is on. Display shows 'Not ready'.
Enabled	Oven operation is enabled. Interior oven light is off. Display shows 'Ready to cook'.
Operation	Oven in operation. Interior oven light is on. Display shows the timer countdown. On completion of cooking, the buzzer is sounded for five seconds. Oven light is on. Display shows 'Cooking complete' while buzzer is sounding.

States and stimuli for the microwave oven (b)

<u>Stimulus</u>	<u>Description</u>
Half power	The user has pressed the half-power button.
Full power	The user has pressed the full-power button.
Timer	The user has pressed one of the timer buttons.
Number	The user has pressed a numeric key.
Door open	The oven door switch is not closed.
Door closed	The oven door switch is closed.
Start	The user has pressed the Start button.
Cancel	The user has pressed the Cancel button.

Model Driven Architecture

- Model driven approach for system design and implementation
- Uses a subset of UML diagrams to design and explain a model
- MDA recommends the use of three types of abstract systems:
 - Computation Independent Model (CIM)
 - Platform Independent Model (PIM)
 - Platform Specific Model (PSM)

Model Driven Architecture

- Computation Independent Model (CIM):
 - Gives the domain abstractions used in a system
 - Therefore, domain models
 - Different CIMs can be used to describe different perspectives of the system

Model Driven Architecture

- Platform Independent Model (PIM):
 - Used to explain the operation of the model
 - The operations described at this abstraction are independent of the platform that might be used to implement the model

Model Driven Architecture

- Platform Specific Model (PSM):
- Transform the PIM to include more specifications in terms of the platform
- Several PSM for one PIM are possible

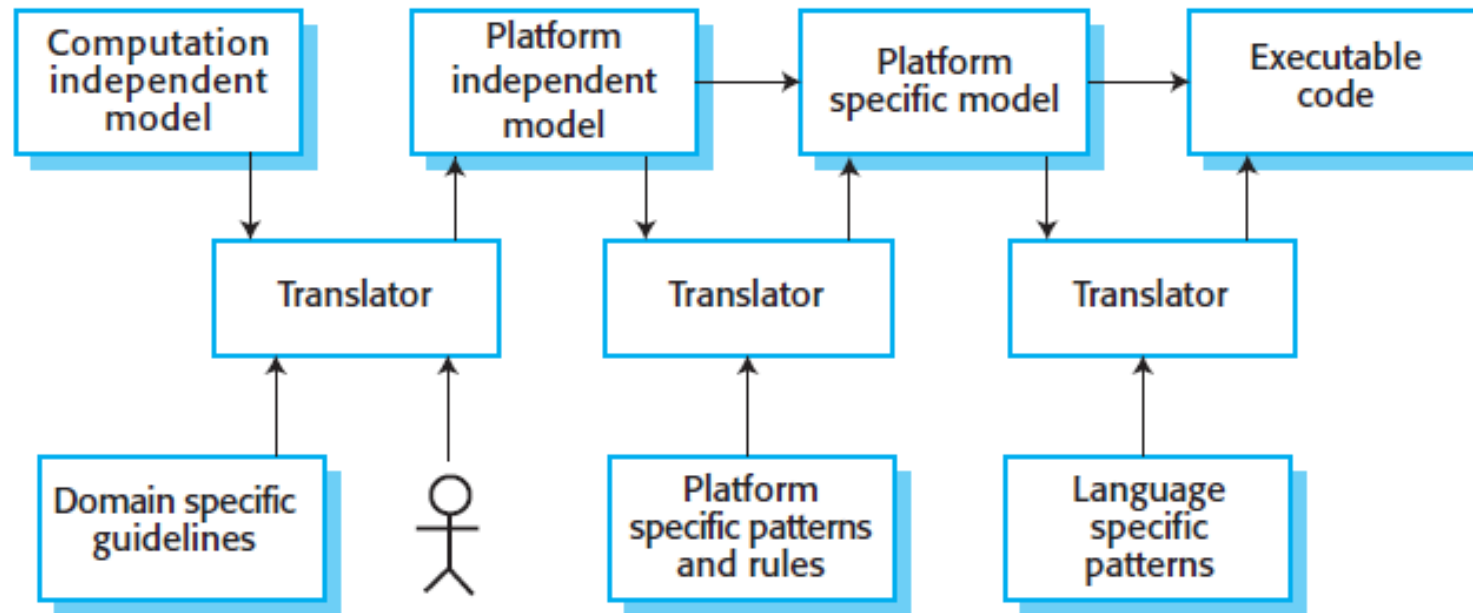
Model Driven Architecture

- Advantages:
- Provides a high-level abstraction of the system
- Reduces the likelihood of errors
- Speeds up design and implementation process
- A single model may be used to obtain system implementations for different platforms

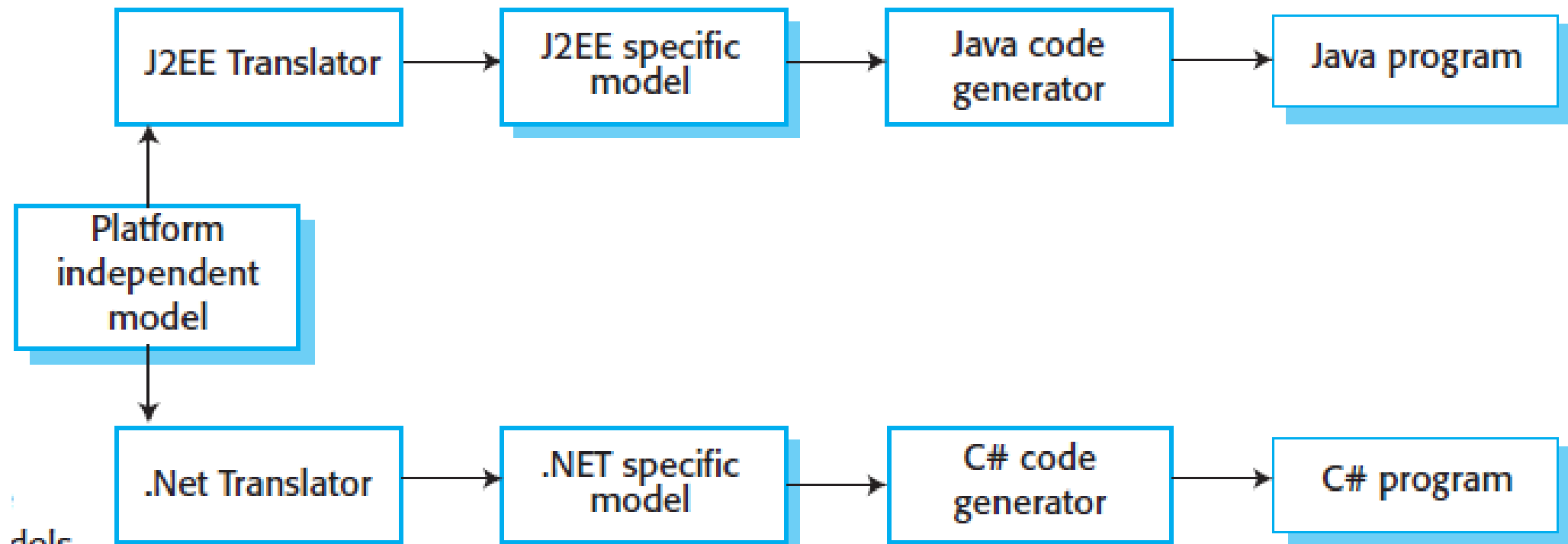
Model Driven Architecture

- Disadvantages:
- Completely automated translation from model to code is not possible
- Linking the concepts using different CIMs is difficult

Model Driven Architecture



Model Driven Architecture



Model Driven Architecture

- Why MDA did not take off as a mainstream approach to software development ?
- As the requirements keep changing, models have to be changed.
- For more complex systems, the use of MDA is limited
- The costs involved in introduction and tooling of MDA are much more than the advantages it provides
- Widespread use of agile methods are diverted attention away from MDA